

A Theoretical Framework for Distribution-Aware Dataset Search

ARYAN ESMAILPOUR, Department of Computer Science, University of Illinois Chicago, USA

SAINYAM GALHOTRA, Department of Computer Science, Cornell University, USA

RAHUL RAYCHAUDHURY, Department of Computer Science, Duke University, USA

STAVROS SINTOS, Department of Computer Science, University of Illinois Chicago, USA

Effective data discovery is a cornerstone of modern data-driven decision-making. Yet, identifying datasets with specific distributional characteristics, such as percentiles or preferences, remains challenging. While recent proposals have enabled users to search based on percentile predicates, much of the research in data discovery relies on heuristic methods, which often result in biased outcomes. This paper presents the first theoretically backed framework that unifies data discovery under centralized and decentralized settings.

More specifically, let $\mathcal{P} = \{P_1, \dots, P_N\}$ be a repository of N datasets, such that each $P_i \subset \mathbb{R}^d$, where d is a constant. We study the percentile-aware indexing (Ptile) problem and the preference-aware indexing (Pref) problem under the centralized and the federated setting. In the centralized setting, we assume direct access to the datasets in $\mathcal{P}$. In the federated setting we are given a synopsis $\mathcal{S}_{P_i}$ which is a compressed representation of P_i that captures the structure of P_i , for every $i \in [N]$. For the Ptile problem, the goal is to construct a data structure such that given a predicate (query rectangle R and an interval θ) report all indexes J such that $j \in J$ if and only if $\frac{|P_j \cap R|}{|P_j|} \in \theta$. For the Pref problem, the goal is to construct a data structure such that given a predicate (query vector $\vec{v}$ and an interval θ) report all indexes J such that $j \in J$ if and only if $\omega_k(P_j, \vec{v}) \in \theta$, where $\omega_k(P_j, \vec{v})$ is the score (inner-product) of the k -th largest projection of P_j on $\vec{v}$. We first show lower bounds for the Ptile and Pref problems in the centralized setting, showing that we cannot hope for near-linear data structures with polylogarithmic query time. Then we focus on approximate data structures for both problems in both settings. We show $\tilde{O}(N)$ space data structures with $\tilde{O}(N)$ preprocessing time, that can answer Ptile and Pref queries in $\tilde{O}(1 + \text{OUT})$ time, where OUT is the output size. The data structures return a set of indexes J such that: i) for every P_i that satisfies the predicate, $i \in J$ and ii) if $j \in J$ then P_j satisfies the predicate up to an additive error of $\varepsilon + 2\delta$, where ε is an arbitrarily small constant and δ is the error of the synopses.

1 Introduction

In today's data-driven world, efficient and effective data discovery [10, 11, 13, 14, 22, 25, 28, 29, 35, 39, 56, 57] plays a pivotal role in unlocking the true potential of information collected from diverse sources. With the rise of deep learning, organizations increasingly rely on vast and varied datasets to drive innovation and gain competitive advantage.

However, existing data discovery systems primarily rely on keyword-based searches or examples to identify relevant datasets [13, 45]. While these methods are ubiquitous, they fall short in scenarios where users have a specific distributional requirement. For instance, users developing machine learning models often require datasets that ensure balanced representation across various groups to prevent overfitting or biases in their models. Furthermore, the problem of selection bias (lack of representative samples) often leads to flawed analyses and unreliable outcomes. These shortcomings are particularly pronounced in industrial settings [20, 40, 51, 52], where datasets are frequently repurposed or reused in contexts different from their original design [9, 19, 30, 59]. Therefore, it is important to develop advanced data discovery techniques that allow users to search for datasets that satisfy their distributional needs. We demonstrate this with the following example.

Authors' Contact Information: Aryan Esmailpour, Department of Computer Science, University of Illinois Chicago, Chicago, USA, aesmai2@uic.edu; Sainyam Galhotra, Department of Computer Science, Cornell University, Ithaca, USA, sg@cs.cornell.edu; Rahul Raychaudhury, Department of Computer Science, Duke University, Durham, USA, rahul.raychaudhury@duke.edu; Stavros Sintos, Department of Computer Science, University of Illinois Chicago, Chicago, USA, stavros@uic.edu.

Example 1.1. Consider an economist who wants to understand the various reasons behind high crime in Brooklyn, NY (more generally, in any specific neighborhood in the US). To ensure statistical significance of the analysis, the economist searches for data that contain a substantial fraction of the data points (say at least 10%) from Brooklyn. More generally, such support requirement would hold even if they want to analyze crime (or other factors affecting development) across different cities across the United States. Several prior works [8, 25, 28, 29] on data discovery have considered such queries for data analysis on open data repositories which contain roughly 100K datasets.

Additionally, the economist may want to identify cities which have at least k neighborhoods with high or low quality of life, modeled as a linear function of crime, pollution, healthcare and other factors that they are interested to consider.

In summary, the economist often faces two types of distributional requirements for data search: (i) The fraction of data points within a specific range (geographical region) must meet or exceed a specified threshold, such as $x\%$. We refer to this query as a percentile-query. (ii) The value (or score) of a specific combination of attributes from at least k data points must exceed a threshold, such as τ . We formalize this notion as a preference query.

The prior work on distribution aware search suffer from two main challenges: First, these techniques cannot handle preference requirements and only support one-sided percentile queries. For example, a query to identify datasets that contain at least 10% points in a rectangular area (that includes Brooklyn) can not be answered with these tools. Second, all these techniques employ heuristics to identify the best datasets. More generally, prior work on data discovery employs heuristics (for example in keyword based data discovery techniques [13, 45]), which do not guarantee the effectiveness of the search algorithm. This means that the search result may miss certain datasets, even though it satisfies user’s query. Missing any dataset that satisfies user query can percolate biases into the system. Missing datasets is generally unacceptable in data marketplaces, where it is extremely crucial to ensure that all datasets that satisfy the query are available to the user. Therefore, it is important to develop a theoretically rigorous framework for data discovery, which guarantees to return all relevant datasets for the user’s query.

In this work, we address these limitations and present the first theoretical framework for data discovery which helps users to identify datasets that satisfy distributional requirements. Next, we describe a new general theoretical framework that is used to formally define various practical problems from distribution-aware dataset search. Later, we show two of the most common subclasses of problems that we focus in this paper, derived from our new framework.

1.1 Theoretical Framework

In this section, we present a new theoretical framework for data discovery.

Dataset. A *schema* $\mathcal{A} = (A_1, A_2, \dots, A_d)$ is an ordered tuple of attributes. A *dataset* P with schema $\mathcal{A}$ is a finite set of data entries, where each entry $p \in P$ is a d -tuple $p = (p_1, p_2, \dots, p_d)$ such that $p_i \in \text{dom}(A_i)$ for $i = 1, 2, \dots, d$. Thus, $P \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_d)$. In this paper, we focus on datasets where all attributes are numerical, i.e., $\text{dom}(A_i) = \mathbb{R}$ for all i . Hence, $P \subset \mathbb{R}^d$. We denote the schema of a dataset P as $\text{SCHEMA}(P)$. In all cases, we assume that $d = O(1)$.

Repository. A *repository* $\mathcal{P}$ is a collection of datasets $\mathcal{P} = \{P_1, P_2, \dots, P_N\}$, all of which share the same schema, i.e., $\text{SCHEMA}(P_i) = \text{SCHEMA}(P_j)$ for all i, j .¹ For each $i \in [N]$, let $n_i = |P_i|$ be the size of dataset P_i , and $N = \sum_{i \in [N]} n_i$.

¹The assumption that all datasets contain only numerical attributes and share the same schema is not strictly required. These assumptions are adopted to simplify the presentation. For our framework to be applicable, it suffices for datasets in $\mathcal{P}$ to share at least some numerical attributes.

Measure Function. A *measure function* $\mathcal{M}(\cdot)$, is a function that, when applied to a dataset P , assigns a numerical value $\mathcal{M}(P) \in \mathbb{R}$ to quantify a specific property or characteristic of the dataset. We assume that $\mathcal{M}(\cdot)$ is applied to a dataset P only if $\mathcal{M}(P)$ is well-defined. For example, a commonly used measure function is the percentile measure function $\mathcal{M}_R$ over a rectangle R , which is defined as $\mathcal{M}_R(P) = \frac{|R \cap P|}{|P|}$.

A set of measure functions that quantify similar properties is called a *class*, which we typically denote by $\mathcal{F}$. For example, a the class of percentile measure functions consists of all measure functions $\mathcal{M}_R$ such as R is any axis-parallel rectangle in $\mathbb{R}^d$.

Predicates. A *predicate* $\text{PRED}(\cdot)$ is a function that, given a dataset P , evaluates to $\text{PRED}(P) \in \{\text{True}, \text{False}\}$. For a measure function $\mathcal{M}$ and an interval $\theta = [a_\theta, b_\theta]$, where $a_\theta, b_\theta \in \mathbb{R}$ and $a_\theta \leq b_\theta$, a *range-predicate* $\text{PRED}_{\mathcal{M},\theta}(P)$ returns True if $\mathcal{M}(P) \in \theta$. If θ is a one-sided interval (for instance, $\theta = [a_\theta, \infty)$), we call $\text{PRED}_{\mathcal{M},\theta}(\cdot)$ a *threshold-predicate*. Complex predicates can be formed by combining individual predicates using conjunction and, disjunctions. For example, consider the complex predicate $\Pi = \text{PRED}_{\mathcal{M},\theta} \wedge \text{PRED}_{\mathcal{M}',\theta'}$, over two measure functions $\mathcal{M}$ and $\mathcal{M}'$ of the same class. The expression $\Pi(P)$ evaluates to True if both $\text{PRED}_{\mathcal{M},\theta}(P)$ and $\text{PRED}_{\mathcal{M}',\theta'}(P)$ are True.

Synopsis. Let $\mathcal{F}$ be a class of measure functions. For a dataset P , a *synopsis* (or *sketch*) with respect to the class $\mathcal{F}$, denoted by $S_P^\mathcal{F}$, is a compressed representation of P that captures the structure of P with regards to the class $\mathcal{F}$. For example, the class of percentile measure functions is defined as the collection of functions $\mathcal{M}_R$ over every rectangle R . Ideally, for any function $\mathcal{M} \in \mathcal{F}$, $\mathcal{M}(P) \approx \mathcal{M}(S_P^\mathcal{F})$, where $\mathcal{M}(S_P^\mathcal{F})$ is the result of evaluating the measure function $\mathcal{M}$ on $S_P^\mathcal{F}$.² We ignore the superscript and only write S_P , when the class $\mathcal{F}$ is clear from context. For a function $\mathcal{M} \in \mathcal{F}$, we denote the error of S_P on $\mathcal{M}$ as $\text{ERR}_{S_P}(\mathcal{M})$, where the specific definition of $\text{ERR}_{S_P}(\cdot)$ depends on the class $\mathcal{F}$ and the dataset P . We define $\text{ERR}_{S_P}(\mathcal{F}) := \max_{\mathcal{M} \in \mathcal{F}} \text{ERR}_{S_P}(\mathcal{M})$.

Distribution-Aware Indexing. Let $\mathcal{P} = \{P_1, P_2, \dots, P_N\}$ be a repository, where each dataset $P_i \subset \mathbb{R}^d$, and let $\mathcal{F}$ be a class of measure functions. The general *distribution-aware indexing problem* for $(\mathcal{P}, \mathcal{F})$ aims to design an index $\mathcal{I}$ that, given a logical expression Π (of constant size) involving predicates of the form $\text{PRED}_{\mathcal{M},\theta}$, where $\mathcal{M} \in \mathcal{F}$ and θ is an interval in $\mathbb{R}$, efficiently computes the subset of datasets in $\mathcal{P}$ that satisfy Π . Formally, the index $\mathcal{I}$ should answer a *distribution-aware dataset query* q_Π :

$$q_\Pi(\mathcal{P}) := \{i \in [N] \mid \Pi(P_i) = \text{True}\}.$$

There are two natural variants of the problem. In the *centralized* setting, we have direct and full access to all the datasets in $\mathcal{P}$ for building $\mathcal{I}$. In the *federated* setting, we do not have direct access to $\mathcal{P}$. Instead, we have access to a collection of synopses $\mathcal{P}_S = \{S_{P_1}, S_{P_2}, \dots, S_{P_N}\}$, where S_{P_i} is a synopsis of P_i with respect to $\mathcal{F}$. We note that if $S_{P_i} = P_i$, for every $i \in [N]$, then this is equivalent to the centralized setting.

It is evident that, $q_\Pi(\mathcal{P})$ can be answered exactly in the centralized setting, where direct access to all datasets in $\mathcal{P}$ is available. In the federated setting, however, the ability to evaluate $\Pi(P_i)$, for any i , critically depends on the quality of the synopsis S_{P_i} provided for each dataset P_i . Specifically, the ability to correctly decide any predicate $\text{PRED}_{\mathcal{M},\theta_j}(P_i)$ is influenced by the error $\text{ERR}_{S_{P_i}}(\mathcal{M})$, which is a measure of deviation between $\mathcal{M}(S_{P_i})$ and $\mathcal{M}(P_i)$.

1.2 Problem definition

In order to define a problem based on the proposed framework in Section 1.1, someone needs to specify the class of measure functions. In this paper, we propose efficient data structures for two of the most common classes of measure functions in the distribution-aware indexing problem.

²We assume $\mathcal{M}(S_P^\mathcal{F})$ is well-defined.

Percentile-aware indexing. Let $P \subset \mathbb{R}^d$ be a dataset and let R be an axis-parallel rectangle in $\mathbb{R}^d$. A *percentile measure function* $\mathcal{M}_R$ is defined as: $\mathcal{M}_R(P) := \frac{|R \cap P|}{|P|}$. Interpreting P as a uniform discrete probability distribution with support defined by the points in P , $\mathcal{M}_R(P)$ measures the mass of the distribution in the region R . A percentile measure function is relevant when the user is interested in the proportion of tuples within a specific region of a dataset, e.g., the proportion of tuples from Brooklyn. We denote the class of axis-parallel percentile measure functions in $\mathbb{R}^d$ by:

$$\mathcal{F}_{\square}^d = \{\mathcal{M}_R(\cdot) \mid R \text{ is an axis-aligned rectangle in } \mathbb{R}^d\}.$$

For the class $\mathcal{F}_{\square}^d$, the commonly used synopses in practice include probability distributions over $\mathbb{R}^d$, such as histograms, mixture models, and ε -samples (a formal definition of an ε -sample is given in the next section). Such synopses support random sampling over the compressed representation of their datasets. The error in this context is defined as $\text{ERR}_{S_{P_i}}(\mathcal{M}_R) := |\mathcal{M}_R(P_i) - \mathcal{M}_R(S_{P_i})|$, where $\mathcal{M}_R(S_{P_i}) := \Pr_{p \sim S_{P_i}}[p \in R]$. For simplicity, we assume that for every $i \in [N]$, $\text{ERR}_{S_{P_i}}(\mathcal{F}_{\square}^d) \leq \delta_i \leq \delta$, where $\delta \in [0, 1)$ is a known arbitrarily small constant. For the class of measure functions $\mathcal{F}_{\square}^d$, the problem derived by our new framework is better known in the literature as *percentile-aware indexing (Ptile)* [6, 8].

PROBLEM 1 (PTILE). Let $\mathcal{P}$ be a repository over a set of (possibly) unknown datasets $\{P_1, \dots, P_N\}$, where $P_i \subset \mathbb{R}^d$ for every $i \in [N]$ and $d = O(1)$. We are given a family of N synopses $\{S_{P_1}, \dots, S_{P_N}\}$, with regards to the class of measure functions $\mathcal{F}_{\square}^d$, such that $\text{ERR}_{S_{P_i}}(\mathcal{F}_{\square}^d) \leq \delta$ for every $i \in [N]$, for $\delta \in [0, 1)$. The goal is to construct a data structure such that, given a logical expression Π over a constant number of range-predicates of the form $\text{PRED}_{\mathcal{M}_R, \theta}$, where $\mathcal{M}_R \in \mathcal{F}_{\square}^d$, it returns $q_{\Pi}(\mathcal{P})$.

In the centralized setting ($\delta = 0$), we call the problem the *centralized percentile-aware indexing (CPtile)*, and in the federated setting ($\delta > 0$), *federated percentile-aware indexing (FPtile)*.

Preference-aware Indexing. Let $P \subset \mathbb{R}^d$ be a dataset. For a unit vector $\vec{v} \in \mathbb{R}^d$ and a point $p \in P$, define the score of point p with respect to vector $\vec{v}$ as $\omega(p, \vec{v}) = \langle p, \vec{v} \rangle$, i.e., the inner product of p with $\vec{v}$. For an integer $k > 0$, let $\omega_k(P, \vec{v})$ denote the score of the point in P with the k -th largest inner-product with respect to $\vec{v}$. A *top- k preference measure function* is defined as: $\mathcal{M}_{\vec{v}, k}(P) := \omega_k(P, \vec{v})$. A top- k preference measure function is relevant when the user is interested in the top- k scores of a dataset with respect to some linear function over the attributes, e.g., cities with k neighborhoods with high/low quality of life, modeled as a linear function. We denote the class of top- k preference measure functions in $\mathbb{R}^d$ by

$$\vec{\mathcal{F}}_k^d = \{\mathcal{M}_{\vec{v}, k}(\cdot) \mid \vec{v} \text{ is a unit-vector in } \mathbb{R}^d\}.$$

For the class $\vec{\mathcal{F}}_k^d$, then a commonly used synopsis in practice include a kernel [5, 37, 55] or a histogram. Such synopses have a mechanism to return $\mathcal{M}_{\vec{v}, k}(S_P)$, i.e., an estimation of the k -th largest score in dataset P with respect to unit vector $\vec{v}$. The error in this context is also defined as $\text{ERR}_{S_P}(\mathcal{M}_{\vec{v}, k}) := |\mathcal{M}_{\vec{v}, k}(P) - \mathcal{M}_{\vec{v}, k}(S_P)|$. For simplicity, we assume that for every $i \in [N]$, $\text{ERR}_{S_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta_i \leq \delta$, where $\delta \in [0, 1)$ is a known arbitrarily small constant. For the class of measure functions $\vec{\mathcal{F}}_k^d$, the problem derived by our new framework is called *preference-aware indexing (Pref)*.

PROBLEM 2 (PREF). Let $\mathcal{P}$ be a repository over a set of (possibly) unknown datasets $\{P_1, \dots, P_N\}$, where $P_i \subset \mathbb{R}^d$ for every $i \in [N]$ and $d = O(1)$. Let k be a positive integer number. We are given a family of N synopses $\{S_{P_1}, \dots, S_{P_N}\}$, with regards to the class of measure functions $\vec{\mathcal{F}}_k^d$, such that $\text{ERR}_{S_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta$ for every $i \in [N]$, for $\delta \in [0, 1)$. The goal is to construct a data structure such that,

given a logical expression Π over a constant number of threshold-predicates of the form $\text{PRED}_{\mathcal{M}_{\vec{v},k},\theta}$, where $\mathcal{M}_{\vec{v},k} \in \tilde{\mathcal{F}}_k^d$, it returns $q_\Pi(\mathcal{P})$.

In the centralized setting ($\delta = 0$), we call the problem *centralized preference-aware indexing (CPref)*, and in the federated setting ($\delta > 0$) *federated preference-aware indexing (FPref)*.

1.3 Our Results

We use $\tilde{O}(\cdot)$ notation to suppress exact dependencies on $\log N$ and $\log \mathcal{N}$. Detailed dependencies are fully specified in subsequent sections. In all cases, we denote the set of reported indexes (corresponding to datasets in $\mathcal{P}$) as J , and sometimes refer to $\text{OUT} = |J|$ as the output size.

- **(Section 3: Lower Bounds)** In Subsection 3.1, we focus on CPtile and rely on the conjectured hardness of the *strong set intersection problem* to show that no data structure with $\tilde{O}(\mathcal{N})$ size and $\tilde{O}(1 + \text{OUT})$ query time exists in $\mathbb{R}^2$, even with a single threshold-predicate (conditional lower bound). In Subsection 3.2, using the hardness of the *halfspace reporting problem*, we show that no data structure of $\tilde{O}(\mathcal{N})$ size and $\tilde{O}(1 + \text{OUT})$ query time exists for the CPref problem in $\mathbb{R}^5$, even with one predicate (unconditional lower bound).

- **(Section 4: Approximate Data structures for Ptile)** We first, design an approximate data structure for the Ptile problem in $\mathbb{R}^d$, supporting a single range-predicate. It has $\tilde{O}(\mathcal{N})$ space, $\tilde{O}(\mathcal{N})$ preprocessing time, and $\tilde{O}(1 + \text{OUT})$ query time. For a query $\Pi = \text{PRED}_{\mathcal{M}_R,\theta}$, with R as a query rectangle and $\theta = [a_\theta, 1]$, the data structure returns J such that $q_\Pi(\mathcal{P}) \subseteq J$, and if $j \in J$, then $a_\theta - \varepsilon - 2\delta \leq \mathcal{M}_R(P_j) \leq b_\theta + \varepsilon + 2\delta$, with high probability, where $\varepsilon \in (0, 1)$ is a small constant. Next, we extend the data structure to work for general intervals $\theta = [a_\theta, b_\theta]$ (range-predicate), having the same error and the same theoretical guarantees. In Appendix C.4, we extend the structure further to handle logical expressions (supporting disjunctions and conjunctions) of m range-predicates, where $m = O(1)$, while retaining the same error bounds and same asymptotic complexities.

- **(Section 5: Approximate Data structures for Pref)** We design an approximate data structure that works for the Pref problem assuming one threshold-predicate. It has $\tilde{O}(\mathcal{N})$ space, $\tilde{O}(\mathcal{N})$ preprocessing time, and $\tilde{O}(1 + \text{OUT})$ query time. For a query $\Pi = \text{PRED}_{\mathcal{M}_{\vec{v},k},\theta}$, where $\vec{v}$ is a query unit vector and θ is a one-sided query interval, it returns a set J such that $q_\Pi(\mathcal{P}) \subseteq J$, and if $j \in J$, then $\mathcal{M}_{\vec{v},k}(P_j) \geq a_\theta - \varepsilon - 2\delta$. In Appendix D.1, we extend this data structure to handle logical expressions of m threshold-predicates, where $m = O(1)$, while retaining the same error bounds and same asymptotic complexities.

- **(Additional Results)** All of our data structures satisfy the following properties: i) They extend to settings where δ_i 's are unknown, with no global upper bound δ , while maintaining the same guarantees. ii) They are dynamic, supporting $\tilde{O}(1)$ update time for the insertion or deletion of a synopsis. iii) They provide $\tilde{O}(1)$ delay guarantees, ensuring the time between reporting two consecutive indexes is $\tilde{O}(1)$.

1.4 Related work

In traditional database systems, there are two settings for query-driven dataset search: data lakes and metadata-based search. Data lakes usually assume full data access and their proposed solutions do not have theoretical guarantees [10, 11, 22, 24, 28, 29, 39, 42, 43, 46]. Existing metadata-based dataset search engines, such as [12, 13], are great engineering solutions but they do not provide theoretical guarantees or/and are limited to keyword search.

Asudeh and Nargesian [6] developed a system vision for distribution-aware data discovery. While their vision targets the data lake setting, it highlights the importance of distributional queries over

dataset collections. Similarly, Nargesian et al. [41] proposed distribution tailoring to meet fairness requirements in the same setting. These methods require raw data access and they do not provide theoretical error guarantees for our studied problems.

The definition of Ptile problem was first given in [8], which built a federated search technique assuming that histograms represent the synopsis of each dataset P_i . While their method detects all datasets whose percentile measure function is in θ , its query time is super linear with respect to the number of datasets. Furthermore, they only studied one-sided query predicates θ and their method does not extend to range predicates.

Our problems for the centralized setting can be modeled as range color reporting queries. Indeed, if every dataset has a different color, then the CPtile problem asks to report all colors in a query rectangle R that contain a fraction (at least a_θ and at most b_θ) of their points in R . Similarly, the CPref problem asks to report all colors such that the score of their k -th larger point on a vector $\vec{v}$ is at least a_θ . The paper closest to our work is [2], where the authors study range summary queries on colored points. Given a set of n colored points $P \subset \mathbb{R}^d$ the goal is to construct a data structure such that given a query rectangle R , it reports all colors that appear at least $\varepsilon|P \cap R|$ times in R . There are fundamental differences with our problem definition and the proposed data structures. i) Our CPtile problem aims to report an index (color) j if $\frac{|P_j \cap R|}{|P_j|} \in \theta$. In their setting they only handle one sided intervals, and j is reported if $|P_j \cap R| \geq \varepsilon \cdot \sum_{i \in [N]} |P_i \cap R|$. It is not clear how the data structures in [2] can be used to solve our problems efficiently. ii) They design data structures only for $d \leq 3$. iii) They do not consider the FPtile or the FPref problems.

Furthermore, there are more works on range queries in colored points such as, color reporting [15, 31], color counting [34], approximate color counting [44, 49], or more complex queries on colors such as entropy queries [23, 36]. However, the techniques of these papers do not apply to our problem. Lu and Tao [38] studied the orthogonal range reporting problem with keywords. Our studied problems are fundamentally different than the problem in [38] and it is not clear how their methods can be used for Ptile and Pref problems under the centralized and federated setting.

2 Preliminaries

Hyper-rectangles. A hyper-rectangle R in $\mathbb{R}^d$ is usually defined in two ways. Either R is defined by its two opposite corners $R^-, R^+ \in \mathbb{R}^d$, or R is defined as the product of d intervals $R = [R_1^-, R_1^+] \times \dots \times [R_d^-, R_d^+]$, where R_j^-, R_j^+ are the j -th coordinates of the points R^-, R^+ , respectively. For $d = 2$, R is a rectangle, and for $d = 1$, R is an interval. For simplicity, for every $d \geq 1$ we call R a rectangle. An open rectangle R that is only defined by one of its corners is called an *orthant*.

ε -sample. Let $\mathcal{S} = (\mathbf{X}, \mathcal{R})$ be a range space, where $\mathbf{X}$ is a set of points in $\mathbb{R}^d$ and $\mathcal{R}$ is a set of ranges in $\mathbb{R}^d$. For $\varepsilon \in [0, 1]$, a subset $C \subseteq \mathbf{X}$ is an ε -sample for $\mathbf{X}$, if for any range $R \in \mathcal{R}$, we have $\left| \frac{|X \cap R|}{|X|} - \frac{|C \cap R|}{|C|} \right| \leq \varepsilon$. It is known by the ε -sample theorem [53], that if $(\mathbf{X}, \mathcal{R})$ is a range space with constant VC dimension, then a random subset $S \subseteq \mathbf{X}$ of cardinality $O(\varepsilon^{-2}(\log(\varepsilon^{-1}) + \log(\varphi^{-1})))$, is an ε -sample for $\mathbf{X}$ with probability at least $1 - \varphi$. If $\mathcal{R}$ is a set of rectangles then the random subset C has cardinality $O(\varepsilon^{-2} \log \varphi^{-1})$, [17, 32]. The next lemma follows straightforwardly by our definitions and properties of ε -samples. Its proof is shown in Appendix A.

LEMMA 2.1. *Let $\mathcal{S}_X$ be the synopsis of X with respect to the class $\mathcal{F}_\square^d$. Assume that $\text{ERR}_{\mathcal{S}_X}(\mathcal{F}_\square^d) = \delta$ for a parameter $\delta \in [0, 1]$. If S is a set of $O(\varepsilon^{-2} \log \varphi^{-1})$ random samples from $\mathcal{S}_X$, then S is an $(\varepsilon + \delta)$ -sample for X with probability at least $1 - \varphi^{-1}$.*

ε -net. Let $\mathbb{S}^{d-1}$ be the unit sphere in $\mathbb{R}^d$. A centrally symmetric set $C \subseteq \mathbb{S}^{d-1}$ (i.e., if $u \in C$, then $-u \in C$) of $O(\varepsilon^{-d+1})$ unit vectors is an ε -net if for every point $v \in \mathbb{S}^{d-1}$, there exists a point $u \in C$ with angle at most $\cos^{-1}\left(\frac{1}{1+\varepsilon^2}\right) = O(\varepsilon)$. An ε -net in $\mathbb{R}^d$ can be constructed in $O(\varepsilon^{-d+1})$ time [3].

Range tree. Let X be a set of finite weighted points in $\mathbb{R}^d$. For a point $x \in X$, let $w_x \in \mathbb{R}$ be its weight, and let $W = \{w_x \mid x \in X\}$ be the list of weights. The goal is to construct a data structure such that given a query rectangle R , and an interval I the set $X_{R,I} = \{x \in X \mid x \in R, w_x \in I\}$ is reported efficiently. A range tree $\mathcal{T}$ constructed on X, W , is a near-linear space tree-based data structure that constructs a hierarchical decomposition of the space such that, for any rectangle R and interval I , all points in $X_{R,I}$ are reported in $O(\text{polylog}(|X| + \text{OUT}))$, where OUT is the output size $|X_{R,I}|$. It is known that range trees over weighted points have $O(|X| \log^d |X|)$ space, $O(|X| \log^d |X|)$ preprocessing time, and can answer range reporting queries in $O(\log^d(|X|) + \text{OUT})$ time. See [21] for details. It is also known how to design dynamic range trees that handle insertions and deletions of points, efficiently. More specifically, a dynamic range tree over weighted points has $O(|X| \log^d |X|)$ space, $O(|X| \log^d |X|)$ preprocessing time, $O(\log^{d+1} |X|)$ update time and can answer range reporting queries in $O(\log^{d+1}(|X|) + \text{OUT})$ time. See [16, 47] for details. Let $\mathcal{T} = \text{DRangeTreeConstruct}(X, W)$ be a method that takes the set of points X along with their weights W and constructs a dynamic range tree, denoted by $\mathcal{T}$. If we construct a static range tree we use $\text{RangeTreeConstruct}(X, W)$. Given a query rectangle R and interval I , let $\mathcal{T}.\text{Report}(R, I)$ be the query procedure that returns $X_{R,I}$. We also define the procedure $\mathcal{T}.\text{ReportFirst}(R, I)$ that reports one (arbitrary) point from $X_{R,I}$, if $X_{R,I} \neq \emptyset$, and then terminate the query procedure. On the other hand, if $X_{R,I} = \emptyset$ then it returns NULL. By slightly modifying the standard query procedures, it is executed in $O(\log^{d+1}(|X|))$ time and $O(\log^d(|X|))$ time for dynamic and static range trees, respectively.

Delay guarantees. A data structure for a reporting (or enumeration) problem over a set X of n points, satisfies $f(n)$ delay (for a real function f) if the time between the start of the reporting procedure to the first result, the time between two consecutive pair of results, and the time between the last result and the termination of the reporting procedure are all bounded by $f(n)$ time [7].

3 Lower Bounds

In this section we show some lower bounds for the Ptile and Pref problems. We first show a conditional lower bound for the CPtile problem using the strong set intersection conjecture. Next, we show a lower bound for the Pref using a known hardness result for halfspace reporting queries. The missing proofs can be found in Appendix B.

3.1 CPtile problem

In this section, we show a conditional lower bound suggesting that no efficient data structure exists for the exact CPtile problem. More precisely, assuming the strong set intersection conjecture, we show that for the CPtile problem even with one threshold-predicate, there is no data structure that uses near linear ($\tilde{O}(\mathcal{N})$) space and answers the queries in near constant ($\tilde{O}(1)$) time even for $d = 2$. The strong set intersection conjecture is a commonly used tool for showing tradeoffs between the space and the query time of the data structures [36, 54, 58], introduced in [27], and can be stated as follows.

Definition 3.1. [Set intersection] Given a universe of integers $\mathcal{U}$, and g subsets $\{S_1, S_2, \dots, S_g\}$, set intersection query asks for reporting $S_i \cap S_j$ for two arbitrary indices $i, j \in [g]$.

CONJECTURE 3.2. *[Strong set intersection conjecture] Given g sets $S_1, \dots, S_g$ with $M = \sum_{i \in [g]} |S_i|$, any index that answers a set intersection query in time $\tilde{O}(\mathbb{T} + \text{OUT})$, needs at least $\tilde{\Omega}(M + \frac{M^2}{\mathbb{T}^2})$ space, where $\mathbb{T}$ is any user defined positive integer parameter and OUT is the size of the output of the query.*

We note that a stronger version of Conjecture 3.2, is stated in [27], however, this weaker conjecture suffices for us to show the desired lower bounds.

To show a reduction from the set intersection problem to our CPtile problem, we first prove a conditional hardness result for a more restricted version of the set intersection problem, which is of independent interest and could be used to show lower bounds for other restrictive problems. We call a collection of sets *uniform* if any element belongs to the same number of different sets. More formally, a collection of sets $T = \{T_1, \dots, T_g\}$ over the universe $\mathcal{U}$ is *uniform* if for any pair of elements $u_1, u_2 \in \mathcal{U}$, we have $|\{T_i \in T : u_1 \in T_i\}| = |\{T_i \in T : u_2 \in T_i\}|$. Furthermore, we define the *uniform set intersection* problem similar to 3.1, but with an additional constraint that the input collection of sets is uniform. The proof of the next lemma can be found in Appendix B.

LEMMA 3.3. *For the uniform set intersection problem with total input size M , there is no data structure of size $\tilde{O}(M)$ that answers queries in time $\tilde{O}(1 + \text{OUT})$, unless the strong set intersection conjecture is false, where OUT is the size of the output of the query.*

In Appendix B.1, we reduce the uniform set intersection problem to the CPtile problem.

THEOREM 3.4. *Given a repository $\mathcal{P} = \{P_1, \dots, P_N\}$ such that $N = \sum_{P_i \in \mathcal{P}} |P_i|$ and $P_i \subset \mathbb{R}^2$ for every $i \in [N]$, there is no data structure for the CPtile problem of size $\tilde{O}(N)$ and $\tilde{O}(1 + \text{OUT})$ query time, unless the strong set intersection conjecture is false.*

3.2 CPref problem

In this subsection, we show an unconditional lower bound for the CPref problem. In the halfspace reporting problem, we are given a set $U \subset \mathbb{R}^d$ of n points and the goal is to construct a data structure such that given a query halfspace H , report all points in $|U \cap H|$. The next lower bound for halfspace reporting queries is known from [1]. Any data structure that answers $d(d+3)/2$ -dimensional halfspace reporting queries in $\tilde{Q}(n) + O(K)$ time, where K is the number of reported points, requires $\Omega\left(\left(\frac{n}{\tilde{Q}(n)}\right)^d 2^{-O(\sqrt{\log \tilde{Q}(n)})}\right)$ space. Hence, there is no hope to get an $\tilde{O}(n)$ space data structure with $\tilde{O}(1)$ query time for the halfspace reporting problem in $\mathbb{R}^d$ for $d > 4$. Due to space limit, we show a reduction from the halfspace reporting problem to CPref problem in Appendix B.2 and we conclude with the next theorem.

THEOREM 3.5. *Given a repository $\mathcal{P} = \{P_1, \dots, P_N\}$ such that $N = \sum_{P_i \in \mathcal{P}} |P_i|$ and $P_i \subset \mathbb{R}^5$, for every $i \in [N]$, there is no data structure for the CPref problem of size $\tilde{O}(N)$ and $\tilde{O}(1 + \text{OUT})$ query time.*

4 Data structures for the Ptile Problem

In this section we describe our data structures for the Ptile problem. In the previous section, we showed that we cannot hope for an efficient exact data structure for the CPtile problem when $d \geq 2$, even if the logical expression contains only one threshold-predicate. In Appendix C.1, we show an efficient exact data structure for the CPtile in $\mathbb{R}^1$ assuming one range-predicate in the logical expression Π . In this section we focus on approximate data structures for the Ptile for any $d \geq 1$. In Subsection 4.1, we show the high level ideas and a technical overview of our new data structures. Then, as a warm-up, in Subsection 4.2, we consider Ptile queries with one threshold predicate. In

Subsection 4.3, we extend the approximate data structure for Ptile queries with a general range-predicate. Finally, in Appendix C.4, we further extend to any logical expression (of disjunctions and conjunctions) Π over a constant number of range predicates. Recall that each synopsis $\mathcal{S}_P$ supports random sampling (with replacement) over P . Let $\mathcal{S}_P.\text{Sample}(\kappa)$ be the synopsis' procedure that returns κ random samples from P , for any positive integer κ .

4.1 Overview

Before we start describing the technical details, we show an overview of our approach. For simplicity we assume that the query logical expression Π consists of only one predicate $\text{PRED}_{\mathcal{M}_R, \theta}$.

Current methods cannot solve Ptile efficiently with theoretical guarantees, even in the centralized setting where every dataset is known. For example, consider the following baseline. For every dataset P_i construct a range tree to answer range counting queries. Given a query predicate $\text{PRED}_{\mathcal{M}_R, \theta}$, the naive solution goes through each dataset P_i and using the range tree it computes $\frac{|R \cap P_i|}{|P_i|}$. If this value belongs in θ then we report i , otherwise we continue with the next dataset. While this algorithm solves the CPtile problem exactly, it has $\Omega(N)$ query time. Another baseline, that works in both centralized and federated settings, is proposed by [8] that first defined the Ptile problem. While their data structure returns a solution for the Ptile problem with bounded error, the query time is $\Omega(N)$ in the worst case.

We design a new data structure with bounded error having query time with polylogarithmic dependency on N . Our new method consists of two core ideas: First, we construct a small summary for each dataset using only the input synopses. Second we construct a data structure over the summaries in a higher dimensional space that allows us to answer percentile queries in sublinear time with respect to N .

One of the main components of our data structure is the notion of *coreset*. Coresets are used in computational geometry, theory and databases, as small summaries that maintain key properties of the original data [3, 33]. More specifically, for every dataset $i \in [N]$ we get a set S_i of roughly $O(\varepsilon^{-2})$ random samples from $\mathcal{S}_{P_i}$. Assuming that all synopses have error at most δ , from Lemma 2.1, we know that S_i is an $(\varepsilon + \delta)$ -sample. By its definition, an ε' -sample acts like a coreset for percentile queries. Notice that every set S_i can be computed in both settings: In the centralized setting we sample directly from the dataset P_i and $\delta = 0$, so S_i is an ε -sample. In the federated setting we sample from the synopsis $\mathcal{S}_{P_i}$ and S_i is an $(\varepsilon + \delta)$ -sample.

While we reduce the size of each dataset P_i from n_i to $O(\varepsilon^{-2})$ our problem is far from being solved. Given a query rectangle R and predicate θ , we still need to go through each set S_i to answer the query. Our main idea is to construct, all possible combinatorially different rectangles with respect to the coreset S_i , for each $i \in [N]$. Interestingly, for each $i \in [N]$, this number is small, because each coreset is small. For each precomputed rectangle ρ , we store the weight $\frac{|\rho \cap S_i|}{|S_i|}$, i.e., the fraction of points from the coreset that lies in this rectangle. Intuitively, given a query rectangle R , we should find for every dataset, among all precomputed rectangles, the one which is the “most similar” to R in order to estimate the fraction of points it contains in R . If the interval θ has the form $[a_\theta, 1]$ (threshold-predicate in Section 4.2) then it turns out that each dataset i that has at least one precomputed rectangle ρ completely inside R with weight at least $a_\theta - \varepsilon - \delta$, should be reported. If the interval θ has the form $[a_\theta, b_\theta]$ (range-predicate in Section 4.3), then it turns out that each dataset, whose largest precomputed rectangle inside R has weight between $a_\theta - \varepsilon - \delta$ and $b_\theta + \varepsilon + \delta$, should be reported.

In order to guarantee sublinear query time, we construct a geometric data structure among all precomputed rectangles over all datasets. For threshold-predicate (Section 4.2), we search among all precomputed rectangles and find any of them that lies completely inside R with weight at least

$a_\theta - \varepsilon - \delta$ (property **P1**). Similarly, for range-predicate (Section 4.3) we search among all precomputed rectangles to find any of them that is maximal inside R (no other larger precomputed rectangle generated from the same coreset lies completely inside R) with weight between $a_\theta - \varepsilon - \delta$ and $b_\theta + \varepsilon + \delta$ (property **P2**). Once we find such a rectangle (that was generated from coreset, say, S_i), we report i , we remove all precomputed rectangles generated from S_i and continue with the same query until we cannot find a rectangle that satisfies property **P1** (threshold-predicate), or property **P2** (range-predicate). In order to efficiently find precomputed rectangles with these properties and to efficiently remove precomputed rectangles, we use a dynamic range tree.

Recall that the range tree is constructed on a set of points, such that given a query rectangle it reports the points inside the query rectangle efficiently. In order to use it for our purpose, in Section 4.2, we map the precomputed rectangles in $\mathbb{R}^d$ (along with their weights) to weighted points in $\mathbb{R}^{2d}$. The new set of (weighted) points in $\mathbb{R}^{2d}$ is stored in the range tree. Given a query rectangle R in $\mathbb{R}^d$, we map it to a new rectangle (orthant) R' in $\mathbb{R}^{2d}$, such that a precomputed rectangle ρ in $\mathbb{R}^d$ lies completely inside R , if and only if, ρ 's mapped point in $\mathbb{R}^{2d}$ lies inside the orthant R' . In Section 4.3 the geometric data structure is more involved and we need to map the precomputed rectangles to points in $\mathbb{R}^{4d}$. We show the details in Section 4.3.

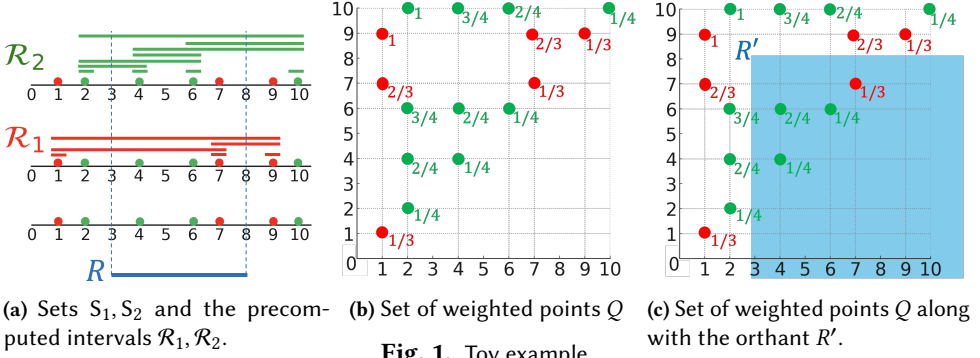


Fig. 1. Toy example.

4.2 Approximate data structure for the Ptile problem with a threshold-predicate

We first design our data structure and prove its guarantees assuming that for every $i \in [N]$, $\text{ERR}_{S_{p_i}}(\mathcal{F}_{\square}^d) \leq \delta_i \leq \delta$, for a known parameter $\delta \in [0, 1)$. In the end, we give the guarantees assuming that δ_i 's are unknown. If $\delta = 0$, then this is the CPtile problem, otherwise it is the FPtile problem. In this subsection, we focus on threshold-predicates, i.e., the query interval θ is defined as $[a_\theta, 1]$.

Data structure. In Algorithm 1, we show the pseudocode of the construction phase. For every $i \in [N]$, we sample a set S_i of $\Theta(\varepsilon^{-2} \log(N\varphi^{-1}))$ points from S_{p_i} . From Lemma 2.1, we know that S_i is a $(\varepsilon + \delta)$ -sample with probability at least φ^{-1} . For every $i \in [N]$, we construct the set $\mathcal{R}_i$ that contains all combinatorially different hyper-rectangles defined by S_i . Every hyper-rectangle $\rho \in \mathcal{R}_i$ is defined by their two opposite corners $\rho^-, \rho^+ \in \mathbb{R}^d$ such that $\rho_h^- \leq \rho_h^+$, for every $h \in [d]$, where ρ_h^-, ρ_h^+ are the h -coordinates of ρ^- and ρ^+ , respectively. Let $q_\rho = (\rho_1^-, \dots, \rho_d^-, \rho_1^+, \dots, \rho_d^+)$ be the point in $\mathbb{R}^{2d}$ defined by merging the opposite corners of ρ . We also store in q_ρ .IndexDataset the index i so that we know the index of the dataset/synopsis that q_ρ was constructed for (it will be useful in the query phase). We set the weight of q_ρ to be $w_{q_\rho} = \mathcal{M}_\rho(S_i) = \frac{|\rho \cap S_i|}{|S_i|}$. Let $Q_i = \{q_\rho \mid \rho \in \mathcal{R}_i\}$ and $W_i = \{w_{q_\rho} \mid q_\rho \in Q_i\}$. Finally, we define $Q = \bigcup_{i \in [N]} Q_i$ and $W = \bigcup_{i \in [N]} W_i$. We construct a dynamic range tree $\mathcal{T}$ over the (weighted) points Q with weights W calling the function $\text{DRangeTreeConstruct}(Q, W)$.

Algorithm 1: ConstructT($\mathcal{S}_{P_1}, \dots, \mathcal{S}_{P_N}, \varepsilon, \varphi$)

```

1  $Q = \emptyset, W = \emptyset;$ 
2 foreach  $i \in [N]$  do
3    $Q_i = \emptyset, W_i = \emptyset;$ 
4    $S_i = \mathcal{S}_{P_i}.\text{Sample}(\Theta(\varepsilon^{-2} \log(N\varphi^{-1})));$ 
5    $\mathcal{R}_i =$  All combinatorially different hyper-rectangles defined by  $S_i$ ;
6   foreach  $\rho \in \mathcal{R}_i$  do
7      $q_\rho = (\rho_1^-, \dots, \rho_d^-, \rho_1^+, \dots, \rho_d^+), q_\rho.\text{IndexDataset} = i, w_{q_\rho} = \frac{|\rho \cap S_i|}{|S_i|};$ 
8      $Q_i = Q_i \cup \{q_\rho\}, W_i = W_i \cup \{w_{q_\rho}\};$ 
9    $Q = Q \cup Q_i, W = W \cup W_i;$ 
10  $\mathcal{T} = \text{DRangeTreeConstruct}(Q, W);$ 
11 return  $\mathcal{T};$ 

```

Algorithm 2: QueryT($\mathcal{T}, Q, R, \theta, \varepsilon, \delta$)

```

1  $R' = [R_1^-, \infty) \times \dots \times [R_d^-, \infty) \times (-\infty, R_1^+] \times \dots \times (-\infty, R_d^+], I' = [a_\theta - \varepsilon - \delta, 1];$ 
2  $q_\rho = \mathcal{T}.\text{ReportFirst}(R', I'), J = \emptyset;$ 
3 while  $q_\rho \neq \text{NULL}$  do
4    $j = q_\rho.\text{IndexDataset}, J = J \cup \{j\}, \text{Report } j;$ 
5   foreach  $q \in Q_j$  do Delete  $q$  from  $\mathcal{T};$ 
6    $q_\rho = \mathcal{T}.\text{ReportFirst}(R', I');$ 
7 foreach  $j \in J$  do Re-insert all points from  $Q_j$  in  $\mathcal{T};$ 
8 return};

```

Example. As an example, we show the construction of our data structure for $d = 1$. We consider two datasets, P_1 and P_2 . In Figure 1a we show the sampled points $S_1 = \{1, 7, 9\}$ and $S_2 = \{2, 4, 6, 10\}$. We construct the sets $\mathcal{R}_1$ and $\mathcal{R}_2$ as shown in Figure 1a containing all combinatorially different rectangles (intervals for $d = 1$) defined by S_1 and S_2 , respectively. For example, $\mathcal{R}_1$ contains the intervals $[1, 1], [7, 7], [9, 9], [1, 7], [1, 9], [7, 9]$ and $\mathcal{R}_2$ contains the intervals $[2, 2], [4, 4], [6, 6], [10, 10], [2, 4], [2, 6], [2, 10], [4, 6], [4, 10], [6, 10]$. Then for each $\rho \in \mathcal{R}_1$ (or $\mathcal{R}_2$) we construct the point $q_\rho \in \mathbb{R}^2$ as shown in Figure 1b. For example, for the interval $\rho = [1, 7]$, we construct the point $q_\rho = (1, 7)$. The weight of point $q_\rho = (1, 7)$ is $2/3$ because $w_{q_\rho} = \frac{|\rho \cap S_1|}{|S_1|} = \frac{2}{3}$. In the end, the dynamic range tree $\mathcal{T}$ is constructed over the weighted points in Figure 1b.

Query procedure. In Algorithm 2, we show the pseudocode of the query phase. We are given a query rectangle R in $\mathbb{R}^d$ and a one-sided query interval $\theta = [a_\theta, 1]$. Let R^-, R^+ be the two opposite corners of R . We define the orthant $R' = [R_1^-, \infty) \times \dots \times [R_d^-, \infty) \times (-\infty, R_1^+] \times \dots \times (-\infty, R_d^+]$ in $\mathbb{R}^{2d}$ and the weight query threshold interval $I' = [a_\theta - \varepsilon - \delta, 1]$. We repeat the following until no point is returned: Using the range tree's $\mathcal{T}$ query procedure $\mathcal{T}.\text{ReportFirst}(R', I')$ we compute, if any, a point q_ρ in $Q \cap R'$ with weight in I' . Let j be the index of the synopsis $\mathcal{S}_{P_j}$ that q_ρ was constructed for. We report j , and we delete from $\mathcal{T}$ all points in Q_j . We continue with the next iteration. If such point q_ρ is not found, we stop the execution. Let J be the indexes we reported. In the end of the query-process, we re-insert in $\mathcal{T}$ all points Q_j for every index $j \in J$.

Example (cont.). We show the query procedure executed on our running example from Figure 1. Without loss of generality, assume $\delta = 0$ (centralized setting) and $\varepsilon < 0.01$.³ Let $R = [3, 8]$ be the query rectangle (interval for $d = 1$), and let $\theta = [0.2, 1]$. We map R to the orthant $R' = [3, \infty) \times (-\infty, 8]$ in $\mathbb{R}^2$ as shown in the blue area of Figure 1c. We also have $I' = [0.2 - \varepsilon, 1]$. Notice that all points in the blue area have weight at least 0.2. Assume that the procedure $\mathcal{T}.\text{ReportFirst}(R', I')$ first returns the point $(4, 4)$. We report the index 2 (because $(4, 4)$ was constructed from S_2). Then we remove from $\mathcal{T}$ all green points. In the next iteration, the procedure $\mathcal{T}.\text{ReportFirst}(R', I')$ returns the point $(7, 7)$ because it lies in the blue orthant and its weight is greater than 0.2. The index 1 is reported and all red points are removed from $\mathcal{T}$. In the end all removed points are re-inserted in $\mathcal{T}$. Both indexes 1 and 2 are correctly reported because $\frac{|R \cap P_1|}{|P_1|} \geq \frac{|R \cap S_1|}{|S_1|} - \varepsilon \geq w_{(4,4)} - \varepsilon > 0.2$ and $\frac{|R \cap P_2|}{|P_2|} \geq \frac{|R \cap S_2|}{|S_2|} - \varepsilon \geq w_{(7,7)} - \varepsilon > 0.2$.

Correctness. Let $\Pi = \text{Pred}_{\mathcal{M}_R, \theta}$ be the logical expression with one threshold-predicate. We first show that we report all the “correct” indexes $q_\Pi(\mathcal{P})$.

LEMMA 4.1. $q_\Pi(\mathcal{P}) \subseteq J$, with probability at least $1 - \varphi$.

PROOF. From Lemma 2.1 we have that every S_i is an $(\varepsilon + \delta)$ -sample for P_i with probability at least $1 - \varphi$. Let $i \in q_\Pi(\mathcal{P})$ be an index such that $a_\theta \leq \mathcal{M}_R(P_i)$. We show that $i \in J$. Let ρ be the largest rectangle in $\mathcal{R}_i$ such that $\rho \subseteq R$. By definition, it holds that $\rho \cap S_i = R \cap S_i$. If we show that $q_\rho \in R' \cap Q$ and $w_{q_\rho} \geq a_\theta - \varepsilon - \delta$, then the result follows. Indeed, by definition of S_i , we have $a_\theta - \varepsilon - \delta \leq \frac{|P_i \cap R|}{|P_i|} - \varepsilon - \delta \leq \frac{|S_i \cap R|}{|S_i|} = \frac{|S_i \cap \rho|}{|S_i|} = \mathcal{M}_\rho(S_i) = w_{q_\rho}$. Furthermore, since $\rho \subseteq R$, we have $R_h^- \leq \rho_h^-$ and $R_h^+ \geq \rho_h^+$ for every $h \in [d]$. Hence, by the definition of the orthant R' , it holds that $q_\rho \in R' \cap Q$. $\square$

LEMMA 4.2. For every $j \in J$ it holds that $\mathcal{M}_R(P_j) \geq a_\theta - 2\varepsilon - 2\delta$, with probability at least $1 - \varphi$.

PROOF. Let $j \in J$ be an index reported by our query procedure. Let $q_\rho \in Q_j$ be the point found in R' at the moment that we reported j . By definition, $w_{q_\rho} = \mathcal{M}_\rho(S_j) = \frac{|\rho \cap S_j|}{|S_j|} \geq a_\theta - \varepsilon - \delta$, and $\rho \subseteq R$. Hence, we have $|\rho \cap S_j| \leq |R \cap S_j|$. By definition of S_j , we have $\mathcal{M}_R(P_j) = \frac{|P_j \cap R|}{|P_j|} \geq \frac{|R \cap S_j|}{|S_j|} - \varepsilon - \delta \geq \frac{|\rho \cap S_j|}{|S_j|} - \varepsilon - \delta \geq a_\theta - 2\varepsilon - 2\delta$. $\square$

Analysis. For simplicity, we assume that ε is an arbitrarily small constant and $\varphi = \frac{1}{N}$. In Appendix C.2 we show the analysis for arbitrary values of ε and φ . The proof of the next lemma can be found in Appendix C.2. We set $\varepsilon \leftarrow \varepsilon/2$ and we get the final result.

LEMMA 4.3. The data structure has $O(N \cdot \log^{4d}(N))$ space and it can be constructed in $O(N \cdot \log^{4d}(N))$ time. The query procedure runs in $O(\log^{2d+1}(N) + |J| \cdot \log^{4d+1}(N))$ time.

THEOREM 4.4. Let $\{S_{P_1}, \dots, S_{P_N}\}$ be the input to the Ptile problem, such that S_{P_i} is a synopsis of a dataset $P_i \subset \mathbb{R}^d$, where $d \geq 1$ is a constant, with $\text{ERR}_{S_{P_i}}(\mathcal{F}_\square^d) \leq \delta$ for $\delta \in [0, 1]$, for every $i \in [N]$. Let $\varepsilon \in (0, 1)$ be an arbitrarily small constant parameter. A data structure of size $O(N \cdot \log^{4d}(N))$ can be constructed in $O(N \cdot \log^{4d}(N))$ time, such that given a query $\Pi = \text{Pred}_{\mathcal{M}_R, \theta}$ for a one-sided interval $\theta = [a_\theta, 1]$, it returns a set of indexes J such that $q_\Pi(\mathcal{P}) \subseteq J$ and for every $j \in J$, $\mathcal{M}_R(P_j) \geq a_\theta - \varepsilon - 2\delta$, with probability at least $1 - \frac{1}{N}$. The query time is $O(\log^{2d+1}(N) + |J| \cdot \log^{4d+1}(N))$.

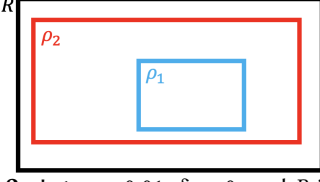


Fig. 2. Let $\varepsilon = 0.01$, $\delta = 0$, and R is the query rectangle. Let $w_{q_{\rho_1}} = \frac{|S_j \cap \rho_1|}{|S|} = 0.1$ and $w_{q_{\rho_2}} = \frac{|S_j \cap \rho_2|}{|S|} = 0.9$, while $\theta = [0, 0.2]$. If the query procedure first finds q_{ρ_1} , it will report the index j . However, the actual measure function of P_j is at least $0.9 - 0.01 \gg 0.2$. Hence j should not be reported.

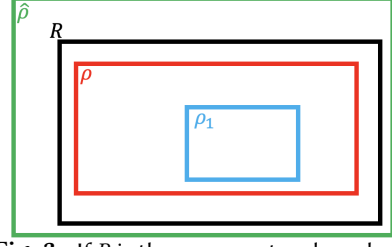


Fig. 3. If R is the query rectangle and ρ is the largest rectangle in R then $\rho \subseteq R \subset \hat{\rho}$. On the other hand, for any other rectangle $\rho_1 \in \mathcal{R}_j$ with $\rho_1 \subset \rho$ there is no point $q_{(\rho_1, \hat{\rho}_1)} \in Q$ such that $R \subset \hat{\rho}_1$ because there exists ρ with $\rho_1 \subset \rho \subseteq R \subset \hat{\rho}_1$.

4.3 Approximate data structure for the Ptile problem with a range-predicate

Unfortunately, the data structure in the previous subsection, does not extend to range-predicates. Previously, any arbitrary precomputed rectangle (with sufficient weight) completely inside R could be selected. However this is not valid for range predicates. For example check Figure 2. Similarly, in the example from Figure 1, assume that $R = [3, 8]$, $\theta = [0.2, 0.4]$, and $\varepsilon < 0.01$. Consider the precomputed interval $[4, 4]$ in $\mathcal{R}_2$. This interval lies inside R and it has weight $1/4 \in [0.2 - \varepsilon, 0.4 + \varepsilon]$. So it is returned by $\mathcal{T}.\text{ReportFirst}(R', I')$. However, $\frac{|R \cap P_2|}{|P_2|} \geq \frac{|R \cap S_2|}{|S_2|} - \varepsilon = w_{(4,6)} - \varepsilon = 2/4 - \varepsilon > 0.4$, so the dataset with index 2 should not be reported.

As we highlighted at the beginning of this section, in order to resolve the issue, we should always consider maximal precomputed rectangles in the data structure. A precomputed rectangle ρ (generated from S_j) is maximal with respect to R if ρ lies inside R and there is no other precomputed rectangle (generated from the same S_j) in R that fully contains ρ . In the example from Figure 1, S_1 's maximal precomputed rectangle (interval in $d = 1$) in R is $[7, 7]$, while S_2 's maximal precomputed interval in R is $[4, 6]$. Hence, only the points $(7, 7)$ and $(4, 6)$ should be considered to answer the query. Indeed, for $\theta = [0.2, 0.4]$, the weight of point $(4, 6)$ is sufficiently larger than 0.4 , so index 2 (correctly) is not reported. On the other hand, the index 1 is reported because the weight of point $(7, 7)$ (that corresponds to the maximal interval $[7, 7]$) lies in $[0.2 - \varepsilon, 0.4 + \varepsilon]$.

In order to search over maximal rectangles, we compute and store all pairs of precomputed rectangles $\rho, \hat{\rho}$ (generated from the same coreset) such that $\rho \subseteq \hat{\rho}$ and there is no other precomputed rectangle ρ' such that $\rho \subset \rho' \subset \hat{\rho}$, where $\rho' \subset \hat{\rho}$ denotes that $\rho' \subset \hat{\rho}$ and the boundary of ρ' does not intersect the boundary of $\hat{\rho}$. Furthermore, we assign the weight $\frac{|\rho \cap S_i|}{|S_i|}$ to the pair $\rho, \hat{\rho}$. We observe that for any query rectangle R , only its maximal precomputed rectangle ρ will belong to at least one pair $(\rho, \hat{\rho})$ such that $\rho \subseteq R \subset \hat{\rho}$. For example, see Figure 3.

From the discussion above, our goal is to construct a data structure such that, given a query rectangle R , and an interval θ , it returns (if it exists) a pair of precomputed rectangles $\rho, \hat{\rho}$ such that $\rho \subseteq R \subset \hat{\rho}$ with weight between $a_\theta - \varepsilon - \delta$ and $b_\theta + \varepsilon + \delta$. Then, remove all pairs of rectangles that contain ρ and continue until no pair is returned. We use a dynamic range tree to find such pairs. Since a range tree stores a set of points, we map every stored pair of precomputed rectangles (along with its weight) to a weighted point in $\mathbb{R}^{4d}$. Given a query rectangle R in $\mathbb{R}^d$, we map it to an orthant R' in $\mathbb{R}^{4d}$ such that there exists a stored pair $\rho, \hat{\rho}$ with $\rho \subseteq R \subset \hat{\rho}$, if and only if, pair's mapped point in $\mathbb{R}^{4d}$ lies inside the orthant R' .

³We note that if $\varepsilon < 0.01$, then we would need more sampled points from each dataset. However, this assumption simplifies the description of the example and successfully derives the main idea of our approach. Another way to think of this is that there exist more samples very far from coordinates $[1, 10]$ that do not play any role in the query $R = [3, 8]$.

Algorithm 3: ConstructR($\mathcal{S}_{P_1}, \dots, \mathcal{S}_{P_N}, \varepsilon, \varphi$)

```

1  $Q = \emptyset, W = \emptyset, \mathcal{B}$ = bounding box;
2 foreach  $i \in [N]$  do
3    $Q_i = \emptyset, W_i = \emptyset$ ;
4    $S_i = \mathcal{S}_{P_i}.\text{Sample}(\Theta(\varepsilon^{-2} \log(N\varphi^{-1})))$ ;
5    $\tilde{S}_i =$  Projections of  $S_i$  onto the  $2 \cdot d$  boundaries of  $\mathcal{B}$ ;
6    $\mathcal{R}_i =$  All combinatorially different hyper-rectangles defined by  $S_i \cup \tilde{S}_i$ ,  $\hat{\mathcal{R}}_i = \emptyset$ ;
7   foreach pair  $(\rho, \hat{\rho}) \in \mathcal{R}_i \times \mathcal{R}_i$  with  $\rho \subseteq \hat{\rho}$  do
8     if  $\nexists \rho' \in \mathcal{R}_i : \rho \subset \rho' \subsetneq \hat{\rho}$  then  $\hat{\mathcal{R}}_i = \hat{\mathcal{R}}_i \cup \{(\rho, \hat{\rho})\}$ ;
9   foreach  $(\rho, \hat{\rho}) \in \hat{\mathcal{R}}_i$  do
10     $q_{(\rho, \hat{\rho})} = (\rho_1^-, \dots, \rho_d^-, \hat{\rho}_1^-, \dots, \hat{\rho}_d^-, \rho_1^+, \dots, \rho_d^+, \hat{\rho}_1^+, \dots, \hat{\rho}_d^+)$ ,  $q_{(\rho, \hat{\rho})}.\text{IndexDataset} = i$ ;
11     $w_{q_{(\rho, \hat{\rho})}} = \frac{|\rho \cap S_i|}{|S_i|}$ ;
12     $Q_i = Q_i \cup \{q_{(\rho, \hat{\rho})}\}$ ,  $W_i = W_i \cup \{w_{q_{(\rho, \hat{\rho})}}\}$ ;
13   $Q = Q \cup Q_i, W = W \cup W_i$ ;
14  $\mathcal{T} = \text{DRangeTreeConstruct}(Q, W)$ ;
15 return  $\mathcal{T}$ ;

```

Algorithm 4: QueryR($\mathcal{T}, Q, R, \theta, \varepsilon, \delta$)

```

1  $R' = [R_1^-, \infty) \times \dots \times [R_d^-, \infty) \times (-\infty, R_1^+) \times \dots \times (-\infty, R_d^+) \times (-\infty, R_1^+) \times \dots \times (-\infty, R_d^+) \times (R_1^+, \infty) \times \dots \times (R_d^+, \infty)$ ;
2  $I' = [a_\theta - \varepsilon - \delta, b_\theta + \varepsilon + \delta]$ ;
3  $q_{(\rho, \hat{\rho})} = \mathcal{T}.\text{ReportFirst}(R', I')$ ,  $J = \emptyset$ ;
4 while  $q_{(\rho, \hat{\rho})} \neq \text{NULL}$  do
5    $j = q_{(\rho, \hat{\rho})}.\text{IndexDataset}$ ,  $J = J \cup \{j\}$ , Report  $j$ ;
6   foreach  $q \in Q_j$  do Delete  $q$  from  $\mathcal{T}$ ;
7    $q_{(\rho, \hat{\rho})} = \mathcal{T}.\text{ReportFirst}(R', I')$ ;
8 foreach  $j \in J$  do Re-insert all points from  $Q_j$  in  $\mathcal{T}$ ;
9 return;

```

Data structure. In Algorithm 3, we show the pseudocode of the construction phase. Without loss of generality assume that all datasets lie in a bounded box $\mathcal{B}$. As we had in Subsection 4.2, for every $i \in [N]$, we get an $(\varepsilon + \delta)$ -sample $S_i \subset \mathbb{R}^d$ by sampling $\Theta(\varepsilon^{-2} \log(N\varphi^{-1}))$ points from $\mathcal{S}_{P_i}$ uniformly at random. For every sample point $s \in S$ we get its projections on the $2 \cdot d$ facets of $\mathcal{B}$, and let $\tilde{S}_i$ be the set of projections.⁴ For every $i \in [N]$, we construct the set $\mathcal{R}_i$ that contains all combinatorially different hyper-rectangles defined by $S_i \cup \tilde{S}_i$. Every hyper-rectangle $\rho \in \mathcal{R}_i$ is defined by their two opposite corners $\rho^-, \rho^+ \in \mathbb{R}^d$ such that $\rho_h^- \leq \rho_h^+$, for every $h \in [d]$, where ρ_h^-, ρ_h^+ are the h -coordinates of ρ^- and ρ^+ , respectively. We construct all pairs of rectangles $\hat{\mathcal{R}}_i = \{(\rho, \hat{\rho}) \mid \rho \in \mathcal{R}_i, \hat{\rho} \in \mathcal{R}_i, \rho \subseteq \hat{\rho}, \nexists \rho' \in \mathcal{R}_i : \rho \subset \rho' \subsetneq \hat{\rho}\}$. For every pair $(\rho, \hat{\rho}) \in \hat{\mathcal{R}}$, let $q_{(\rho, \hat{\rho})} = (\rho_1^-, \dots, \rho_d^-, \hat{\rho}_1^-, \dots, \hat{\rho}_d^-, \rho_1^+, \dots, \rho_d^+, \hat{\rho}_1^+, \dots, \hat{\rho}_d^+)$ be the point in $\mathbb{R}^{4d}$ defined by merging the opposite corners of ρ and $\hat{\rho}$. We also store in $q_{(\rho, \hat{\rho})}.\text{IndexDataset}$ the index i so that we know the index of the dataset/synopsis that $q_{(\rho, \hat{\rho})}$ was constructed for. We set the weight of $q_{(\rho, \hat{\rho})}$ to be $w_{q_{(\rho, \hat{\rho})}} = \mathcal{M}_\rho(S_i) = \frac{|\rho \cap S_i|}{|S_i|}$. Notice that the weight $w_{q_{(\rho, \hat{\rho})}}$ is defined only with respect to ρ . Let

⁴The bounding box $\mathcal{B}$ and the projections $\bigcup_{i \in [N]} \tilde{S}_i$ are needed only in this subsection for technical reasons in the correctness proof.

$Q_i = \{q_{(\rho, \hat{\rho})} \mid (\rho, \hat{\rho}) \in \hat{\mathcal{R}}_i\}$ and $W_i = \{w_{q_{(\rho, \hat{\rho})}} \mid q_{(\rho, \hat{\rho})} \in Q_i\}$. Finally, we define $Q = \bigcup_{i \in [N]} Q_i$ and $W = \bigcup_{i \in [N]} W_i$. We construct a dynamic range tree $\mathcal{T}$ over the (weighted) points Q with weights W calling the function $\text{DRangeTreeConstruct}(Q, W)$.

Example. Back to our example from Figure 1. We construct all pairs in $\hat{\mathcal{R}}_1$ and $\hat{\mathcal{R}}_2$. For instance, the pair $([7, 7], [1, 9])$ belongs in $\hat{\mathcal{R}}_1$ because $[7, 7]$ is a subset of $[1, 9]$ and there is no other interval in $\mathcal{R}_1$ strictly inside $[1, 9]$ that contains $[7, 7]$ (while $[7, 7] \subset [7, 9] \subset [1, 9]$ it does not hold that $[7, 9] \subset [1, 9]$). Similarly, the pair $([4, 6], [2, 10])$ belongs in $\hat{\mathcal{R}}_2$. On the other hand the pair $([6, 6], [2, 10]) \notin \hat{\mathcal{R}}_2$ because $[6, 6] \subset [4, 6] \subset [2, 10]$. Each pair in $\hat{\mathcal{R}}_1 \cup \hat{\mathcal{R}}_2$ is mapped to a weighted point in $\mathbb{R}^4$, for example the pair $([7, 7], [1, 9])$ is mapped to the point $(7, 1, 7, 9) \in \mathbb{R}^4$ with weight $\frac{|[7, 7] \cap S_1|}{|S_1|} = \frac{1}{3}$. The dynamic range tree is constructed over the weighted points in $\mathbb{R}^4$.

Query procedure. In Algorithm 4, we show the pseudocode of the query phase. We are given a query rectangle R in $\mathbb{R}^d$ and a query interval $\theta = [a_\theta, b_\theta]$. Without loss of generality, assume that the query rectangle R is contained in the bounding box $\mathcal{B}$. Let R^-, R^+ be the two opposite corners of R . Without loss of generality we can assume that there is no point from Q that lies on an axis-align hyperplane that is tangent to R . We define the orthant $R' = [R_1^-, \infty) \times \dots \times [R_d^-, \infty) \times (-\infty, R_1^+) \times \dots \times (-\infty, R_d^+) \times (-\infty, R_1^+) \times \dots \times (-\infty, R_d^+) \times (R_1^+, \infty) \times \dots \times (R_d^+, \infty)$ in $\mathbb{R}^{4d}$ and the weight query threshold interval $I' = [a_\theta - \varepsilon - \delta, b_\theta + \varepsilon + \delta]$. We repeat the following until no index is returned: Using the range tree's $\mathcal{T}$ query procedure $\mathcal{T}.\text{ReportFirst}(R', I')$ we compute, if any, a point $q_{(\rho, \hat{\rho})}$ in $Q \cap R'$ with weight in I' . Let j be the index of the synopsis $\mathcal{S}_{P_j}$ that $q_{(\rho, \hat{\rho})}$ was constructed for. We report j , and we delete from $\mathcal{T}$ all points $\{q_{(\rho, \hat{\rho})} \mid \rho \in \mathcal{R}_j\}$. We continue with the next iteration. Let J be the indexes we reported. In the end of the process, we re-insert in $\mathcal{T}$ all points we removed from Q_j for every index $j \in J$.

Example (cont.). Let $\delta = 0$, $\varepsilon < 0.01$, $R = [3, 8]$ and $\theta = [0.2, 0.4]$. We define the orthant $R' = [3, \infty) \times (-\infty, 3) \times (-\infty, 8] \times (8, \infty)$ and the interval $I' = [0.2 - \varepsilon, 0.4 + \varepsilon]$. The procedure $\mathcal{T}.\text{ReportFirst}(R', I')$ returns the point $(7, 1, 7, 9)$ because it lies in R' and its weight is $1/3 \in [0.2 - \varepsilon, 0.4 + \varepsilon]$. Hence, the index 1 is reported. On the other hand, the index 2 is not reported. The only point from Q_2 that lies in R' is the point $(4, 2, 6, 10)$ that corresponds to the pair $([4, 6], [2, 10]) \in \hat{\mathcal{R}}_2$. However, the weight of this point is $\frac{|[4, 6] \cap S_2|}{|S_2|} = \frac{2}{4} = 0.5 > 0.4 + \varepsilon$ so it is not returned. Indeed $\frac{|R \cap P_2|}{|P_2|} \geq \frac{|R \cap S_2|}{|S_2|} - \varepsilon = 0.5 - \varepsilon > 0.4$, so index 2 should not be reported.

Correctness. We first start with a primitive lemma. It shows that whenever $\mathcal{T}$ finds a point $q_{(\rho, \hat{\rho})}$ (with $\rho \in \mathcal{R}_j$), then ρ is actually the largest rectangle in $\mathcal{R}_j$ with $\rho \subseteq R$. See Figure 3.

LEMMA 4.5. *Let $j \in J$ be an index reported by the query procedure and let $q_{(\rho, \hat{\rho})}$ be the point that was found when j was reported. There is no rectangle $\rho' \in \mathcal{R}_j$ such that $\rho \subset \rho' \subseteq R$.*

PROOF. Assume that there exists $\rho' \in \mathcal{R}_j$ such that $\rho \subset \rho' \subseteq R$. By the definition of the query orthant R' , we have $\rho \subseteq R \subset \hat{\rho}$. Hence, we have $\rho \subset \rho' \subset \hat{\rho}$. This is a contradiction because, by the definition of Q_j , a point $q_{(\rho, \hat{\rho})}$ exists if there is no $\rho'' \in \mathcal{R}_j$ with $\rho \subset \rho'' \subset \hat{\rho}$. $\square$

LEMMA 4.6. *If $\rho \in \mathcal{R}_j$ is the maximal rectangle among rectangles in $\mathcal{R}_j$, in the query rectangle R , then there exists $\hat{\rho}^R \in \mathcal{R}_j$ such that $q_{(\rho, \hat{\rho}^R)} \in Q_j$, and $R \subset \hat{\rho}^R$.*

PROOF. Since there is no point in Q that lies on an axis-align hyperplane that is tangent to R we have that $\rho \subset R$. For every facet of R we repeat: we expand the facet until we hit a point from $S_j \cup \hat{S}_j$. Notice that we will always hit such a point, either in S_j or on the boundary $\mathcal{B}$ (if we hit the boundary of $\mathcal{B}$, then by the construction of $\hat{S}_j$ there will be at least a point from $\hat{S}_j$ on the intersection of $\mathcal{B}$ and the extended facet). Let $\hat{\rho}^R$ be the new rectangle. By the construction of $\hat{\rho}^R$

and the fact that ρ is maximal, we have that $R \subset \hat{\rho}^R, \hat{\rho}^R \in \mathcal{R}_j$, and there is no $\rho' \in \mathcal{R}_j$ such that $\rho \subset \rho' \subset \hat{\rho}^R \in \mathcal{R}_j$. $\square$

Let $\Pi = \text{Pred}_{\mathcal{M}_R, \theta}$ be the logical expression with one range-predicate. We show that we report all the “correct” indexes $q_\Pi(\mathcal{P})$.

LEMMA 4.7. $q_\Pi(\mathcal{P}) \subseteq J$, with probability at least $1 - \varphi$.

PROOF. From Lemma 2.1, we have that every S_i is an $(\varepsilon + \delta)$ -sample for P_i with probability at least $1 - \varphi$. Let $i \in q_\Pi(\mathcal{P})$ be an index such that $a_\theta \leq \mathcal{M}_R(P_i) \leq b_\theta$. We show that $i \in J$. Let ρ be the largest/maximal rectangle in $\mathcal{R}_i$ such that $\rho \subseteq R$. By definition, it holds that $\rho \cap S_i = R \cap S_i$. From Lemma 4.6, there exists a point $q_{(\rho, \hat{\rho}^R)} \in Q_i \subseteq Q$. Hence, if we show that $q_{(\rho, \hat{\rho}^R)} \in R' \cap Q$ and $a_\theta - \varepsilon - \delta \leq w_{q_{(\rho, \hat{\rho}^R)}} \leq b_\theta + \varepsilon + \delta$, then the result follows. Indeed, by definition of S_i , we have $a_\theta - \varepsilon - \delta \leq \frac{|P_i \cap R|}{|P_i|} - \varepsilon - \delta \leq \frac{|S_i \cap R|}{|S_i|} = \frac{|S_i \cap \rho|}{|S_i|} = \mathcal{M}_\rho(S_i) = w_{q_{(\rho, \hat{\rho}^R)}}$ and $b_\theta + \varepsilon + \delta \geq \frac{|P_i \cap R|}{|P_i|} + \varepsilon + \delta \geq \frac{|S_i \cap R|}{|S_i|} = \frac{|S_i \cap \rho|}{|S_i|} = \mathcal{M}_\rho(S_i) = w_{q_{(\rho, \hat{\rho}^R)}}$. Furthermore, from Lemma 4.6, $\rho \subseteq R$ and $R \subset \hat{\rho}^R$ so we have $R_h^- \leq \rho_h^-, R_h^- > \hat{\rho}_h^{R-}, R_h^+ \geq \rho_h^+$, and $R_h^+ < \hat{\rho}_h^{R+}$, for every $h \in [d]$. Hence, by the definition of the orthant R' , it holds that $q_{(\rho, \hat{\rho}^R)} \in R' \cap Q$. $\square$

LEMMA 4.8. For every $j \in J$ it holds that $a_\theta - 2\varepsilon - 2\delta \leq \mathcal{M}_R(P_j) \leq b_\theta + 2\varepsilon + 2\delta$, with probability at least $1 - \varphi$.

PROOF. Let $j \in J$ be an index reported by our query procedure. Let $q_{(\rho, \hat{\rho})} \in Q_j$ be the point found in R' at the moment we reported j . By definition, $w_{q_{(\rho, \hat{\rho})}} = \mathcal{M}_\rho(S_j) = \frac{|\rho \cap S_j|}{|S_j|} \geq a_\theta - \varepsilon - \delta$, and $w_{q_{(\rho, \hat{\rho})}} = \frac{|\rho \cap S_j|}{|S_j|} \leq b_\theta + \varepsilon + \delta$, and from Lemma 4.5, we know that ρ is the largest rectangle inside R , so $|\rho \cap S_j| = |R \cap S_j|$. By definition of S_j , we have $\mathcal{M}_R(P_j) = \frac{|P_j \cap R|}{|P_j|} \geq \frac{|R \cap S_j|}{|S_j|} - \varepsilon - \delta = \frac{|\rho \cap S_j|}{|S_j|} - \varepsilon - \delta \geq a_\theta - 2\varepsilon - 2\delta$, and $\mathcal{M}_R(P_j) = \frac{|P_j \cap R|}{|P_j|} \leq \frac{|R \cap S_j|}{|S_j|} + \varepsilon + \delta = \frac{|\rho \cap S_j|}{|S_j|} + \varepsilon + \delta \leq b_\theta + 2\varepsilon + 2\delta$. $\square$

LEMMA 4.9. The query procedure does not report duplicates.

Analysis. For simplicity, we assume that ε is an arbitrarily small constant and $\varphi = \frac{1}{N}$. In Appendix C.3 we show the analysis for arbitrary values of ε and φ . The proof of the next lemma can be found in Appendix C.3.

LEMMA 4.10. The data structure has $O(N \cdot \log^{8d}(N))$ space and it can be constructed in $O(N \cdot \log^{8d}(N))$ time. The query procedure runs in $O(\log^{4d+1}(N) + |J| \cdot \log^{6d+1}(N))$ time.

THEOREM 4.11. Let $\{S_{P_1}, \dots, S_{P_N}\}$ be the input to the Ptile problem, such that S_{P_i} is a synopsis of a dataset $P_i \subset \mathbb{R}^d$, where $d \geq 1$ is a constant, with $\text{ERR}_{S_{P_i}}(\mathcal{F}_\square^d) \leq \delta$ for $\delta \in [0, 1)$, for every $i \in [N]$. Let $\varepsilon \in (0, 1)$ be an arbitrarily small constant parameter. A data structure of size $O(N \cdot \log^{8d}(N))$ can be constructed in $O(N \cdot \log^{8d}(N))$ time, such that given a query $\Pi = \text{Pred}_{\mathcal{M}_R, \theta}$, it returns a set of indexes J such that $q_\Pi(\mathcal{P}) \subseteq J$ and for every $j \in J$, $a_\theta - \varepsilon - 2\delta \leq \mathcal{M}_R(P_j) \leq b_\theta + \varepsilon + 2\delta$, with probability at least $1 - \frac{1}{N}$. The query time is $O(\log^{4d+1}(N) + |J| \cdot \log^{6d+1}(N))$.

Remark 1. The data structure from Theorem 4.11 can be made dynamic under insertion or deletion of synopses. Using the update procedure of the dynamic range tree, if a synopsis S_P is inserted/deleted we can update our data structure in $O(\log^{8d+1}(N))$ time.

Remark 2. If we do not know an upper bound δ on $\text{ERR}_{S_{P_i}}(\mathcal{F}_\square^d)$, then by slightly modifying the data structure above we can get the following result. Let $\text{ERR}_{S_{P_i}}(\mathcal{F}_\square^d) \leq \delta_i$ for unknown parameters $\delta_i \in [0, 1)$. Having the same complexities as in Theorem 4.11, we report a set of indexes J such that, if $a_\theta + \varepsilon + \delta_i \leq \mathcal{M}_R(P_i) \leq b_\theta - \varepsilon - \delta_i$, then $i \in J$, and if $j \in J$, then $a_\theta - \varepsilon - \delta_j \leq \mathcal{M}_R(P_j) \leq b_\theta + \varepsilon + \delta_j$.

Algorithm 5: ConstructTop($\mathcal{S}_{P_1}, \dots, \mathcal{S}_{P_N}, \varepsilon$)

```

1  $C \leftarrow \varepsilon$ -net in  $\mathbb{R}^d$ ;
2 foreach  $\vec{v} \in C$  do
3    $\Gamma_{\vec{v}} = \emptyset$ ;
4   foreach  $i \in [N]$  do
5      $\gamma_{\vec{v}}^{(i)} = \mathcal{S}_{P_i}.\text{Score}(\vec{v}, k), \Gamma_{\vec{v}} = \Gamma_{\vec{v}} \cup \{\gamma_{\vec{v}}^{(i)}\}$ ;
6    $\mathcal{T}_{\vec{v}} = \text{RangeTreeConstruct}(\Gamma_{\vec{v}})$ ;
7  $\mathcal{T} = \bigcup_{\vec{v} \in C} \mathcal{T}_{\vec{v}}$ ;
8 return;

```

Algorithm 6: QueryTop($\mathcal{T}, \vec{u}, \theta, \varepsilon, \delta$)

```

1  $\vec{v} = \arg \min_{\vec{h} \in C} \|\vec{u} - \vec{h}\|$ ;
2  $I' = [a_\theta - \varepsilon - \delta, \infty)$ ;
3  $\Gamma = \mathcal{T}_{\vec{v}}.\text{Report}(I')$ ;
4 foreach  $\gamma_{\vec{v}}^{(j)} \in \Gamma$  do Report  $j$ ;
5 return;

```

Remark 3. We note that the data structure we design satisfy delay guarantees as we report the indexes J , because range tree satisfies delay guarantees and we do not report duplicates. It is only in the end of the query procedure that we need to re-insert all the deleted points from $\mathcal{T}$. Using lazy updates [47, 48] similarly to [4], our data structure has bounded $O(\log^{6d+1}(N))$ delay.

5 Data structures for the Pref Problem

In this section, we design data structures for the Pref problem. Recall that the Pref problem is defined on threshold-predicates over an interval θ of the form $[a_\theta, 1]$. Next, we show an approximate data structure for the Ptile problem assuming only one predicate. In Appendix D.1 we extend to an approximate data structure for the Pref problem considering a logical expression over $m = O(1)$ predicates. Similarly to the previous section, we assume that for every $i \in [N]$, $\text{Err}_{\mathcal{S}_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta_i \leq \delta$, for a known parameter $\delta \in [0, 1)$. In the end, we give the guarantees assuming that δ_i 's, are unknown. For simplicity, we assume that all points in $\bigcup_{i \in [N]} P_i$ lie in the unit ball. Recall that each synopsis $\mathcal{S}_P$ has a mechanism to return $\mathcal{M}_{\vec{v}, k}(\mathcal{S}_P)$, i.e., an estimation of the k -th largest score in P with respect to unit vector $\vec{v}$. Let $\mathcal{S}_P.\text{Score}(\vec{v}, k)$ be the synopsis' procedure that returns $\mathcal{M}_{\vec{v}, k}(\mathcal{S}_P)$.

Overview. The main idea follows from the data structure we designed for the Ptile problem. If we knew the query vector $\vec{u}$, then the Pref would be straightforward. Recall that for the Ptile problem, we did not know the query rectangle R upfront, so the first step was to create all possible combinatorially different rectangles generated from a set of samples. For the Pref, we do not have samples, however, we construct a coreset using the notion of ε -net. While we do not know the query vector $\vec{u}$, by constructing an ε -net we get a set of vectors (of small size) such that for any query unit vector $\vec{u}$ there is always a unit vector in the ε -net that lies very close to $\vec{u}$. While there are infinite query vectors, the intuition is that we can approximate the score of a point p on a query vector $\vec{u}$ by computing the score of p with respect to the vector of the ε -net which is closest to $\vec{u}$. Hence, in the preprocessing phase, we use the synopsis of each dataset to estimate the score of the k -th highest score on every vector in the ε -net. For each vector in the ε -net, we store all the estimated costs (one from each dataset) into a geometric data structure. The computed estimated scores over all the vectors in the ε -net consists of a coreset for our problem. Given a query vector $\vec{u}$ and a one-sided interval $\theta = [a_\theta, 1]$, we find the closest vector in the ε -net and using the corresponding data structure we identify all datasets with scores in the range $[a_\theta - \varepsilon - \delta, 1]$. Because of the coreset construction, this technique returns a solution with theoretical guarantees for the Pref problem.

Data structure. In Algorithm 5, we show the pseudocode of the construction phase. We construct an ε -net C on $\mathbb{R}^d$. For every $\vec{v} \in C$ and every $i \in [N]$, we get the value $\gamma_{\vec{v}}^{(i)} = \mathcal{S}_{P_i}.\text{Score}(\vec{v}, k)$ as an

approximation to $\omega_k(P_i, \vec{v})$. Let $\Gamma_{\vec{v}} = \{\gamma_{\vec{v}}^{(i)} \mid i \in [N]\}$. For every $\vec{v} \in C$, we construct a binary search tree (1-dimensional static range tree) $\mathcal{T}_{\vec{v}}$ on $\Gamma_{\vec{v}}$.⁵

Query procedure. In Algorithm 6, we show the pseudocode of the query phase. Let $\vec{u}$ be the query vector and θ be the one-sided query interval. We go through each vector in C and we compute $\vec{v} = \arg \min_{\vec{h} \in C} \|\vec{u} - \vec{h}\|$. Using the tree $\mathcal{T}_{\vec{v}}$ we report all indexes $j \in [N]$ such that $\gamma_{\vec{v}}^{(j)} \geq a_\theta - \varepsilon - \delta$. Let J be the set of all indexes reported.

Correctness. Let $\Pi = \text{Pred}_{\mathcal{M}_{\vec{u},k},\theta}$ be the logical expression with one threshold-predicate. Recall that $\mathbf{q}_\Pi(\mathcal{P})$ denotes the correct set of indexes we should report.

LEMMA 5.1 ([5]). *For any point p in the unit ball ($\|p\| \leq 1$) and any pair of unit vectors $\vec{v}_1, \vec{v}_2$ such that $\|\vec{v}_1 - \vec{v}_2\| \leq \varepsilon$, $|\omega(p, \vec{v}_1) - \omega(p, \vec{v}_2)| \leq \varepsilon$.*

LEMMA 5.2. $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$ and for every $j \in J$, $\omega_k(P_j, \vec{u}) \geq a_\theta - 2\varepsilon - 2\delta$.

PROOF. Since $\mathcal{S}_{P_i}$ is a synopsis for P_i , by definition we have that $\omega_k(P_i, \vec{v}) - \delta \leq \gamma_{\vec{v}}^{(i)} \leq \omega_k(P_i, \vec{v}) + \delta$ for every $i \in [N]$. First, we assume that an index $i \in \mathbf{q}_\Pi(\mathcal{P})$, so $\omega_k(P_i, \vec{u}) \geq a_\theta$. We show that $i \in J$. By Lemma 5.1, we have $\omega_k(P_i, \vec{v}) \geq \omega_k(P_i, \vec{u}) - \varepsilon \geq a_\theta - \varepsilon$. By the definition of $\gamma_{\vec{v}}^{(i)}$, we have $\gamma_{\vec{v}}^{(i)} \geq \omega_k(P_i, \vec{v}) - \delta \geq a_\theta - \varepsilon - \delta$. Hence i will be reported by the query procedure. Next, we show that for every index $j \in J$, it holds $\omega_k(P_j, \vec{u}) \geq a_\theta - 3\varepsilon - \delta$. The index j is reported because $\gamma_{\vec{v}}^{(j)} \geq a_\theta - \varepsilon - \delta$. By the definition of $\gamma_{\vec{v}}^{(j)}$, we have $\gamma_{\vec{v}}^{(j)} \leq \omega_k(P_j, \vec{v}) + \delta$. Furthermore, from Lemma 5.1, $\omega_k(P_j, \vec{u}) \geq \omega_k(P_j, \vec{v}) - \varepsilon$. Hence, $\omega_k(P_j, \vec{u}) \geq \omega_k(P_j, \vec{v}) - \varepsilon \geq \gamma_{\vec{v}}^{(j)} - \varepsilon - \delta \geq a_\theta - 2\varepsilon - 2\delta$. $\square$

Analysis. For simplicity, we assume that ε is an arbitrarily small constant. In Appendix C.3 we show the analysis for arbitrary values of ε . Let $\Lambda_{\mathcal{S}_{P_i}}$ be the running time of $\mathcal{S}_{P_i}.\text{Score}(\vec{v}, k)$. The proof of the next lemma can be found in Appendix D. We set $\varepsilon \leftarrow \varepsilon/2$ and we get the final result.

LEMMA 5.3. *The data structure has $O(N)$ space and can be constructed in $O(N \log(N) + \sum_{i \in [N]} \Lambda_{\mathcal{S}_{P_i}})$ time. The query time is $O(\log(N) + |J|)$.*

THEOREM 5.4. *Let $\{\mathcal{S}_{P_1}, \dots, \mathcal{S}_{P_N}\}$ be the input to the Pref problem, such that $\mathcal{S}_{P_i}$ is a synopsis of a dataset $P_i \subset \mathbb{R}^d$, where $d \geq 1$ is a constant, with $\text{ERR}_{\mathcal{S}_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta$ for $\delta \in [0, 1)$ and $k \geq 1$, for every $i \in [N]$. Let $\varepsilon \in (0, 1)$ be an arbitrarily small constant parameter. A data structure of size $O(N)$ can be constructed in $O(N \log(N) + \sum_{i \in [N]} \Lambda_{\mathcal{S}_{P_i}})$ time, such that given a query $\Pi = \text{Pred}_{\mathcal{M}_{\vec{u},k},\theta}$ for a one-sided interval $\theta = [a_\theta, 1]$, it returns a set of indexes J such that $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$ and for every $j \in J$, $\mathcal{M}_{\vec{u},k}(P_j) \geq a_\theta - \varepsilon - 2\delta$. The query time is $O(\log(N) + |J|)$.*

6 Future work

There are multiple interesting open problems arising from our data structures and the proposed framework in general.

- Our new framework can be used to define multiple types of distribution-aware queries such as nearest neighbor or diversity queries both in the centralized and federated setting. We are given a repository $P_1, \dots, P_N$. For nearest neighbor queries: given a query point q and a threshold τ , return all datasets P_j such that $\text{dist}(q, P_j) \leq \tau$, where $\text{dist}(q, P_j)$ is the closest distance between q and a point in P_j . For diversity queries: given a query rectangle R and a threshold τ , return all datasets P_j such that $\text{div}(P_j \cap R) \geq \tau$, where $\text{div}()$ is a diversity function [33]. While the goal of the paper is not to provide a data structure with theoretical guarantees for every problem arising from our proposed framework, our techniques are useful to handle additional queries such as nearest neighbor and diversity queries. The main obstacle for these queries is that, to the best of our knowledge, there are no high-quality and small-size coresets we can use for the construction of our data structures. Recall

⁵Since $\gamma_{\vec{v}}^{(i)} \in \mathbb{R}$, we call it either a point in $\mathbb{R}^1$ or a score/value.

that for the Ptile problem we used the notion of ε -sample, while for the Pref problem we used the notion of ε -net to construct coresets that are sufficient to answer any percentile or preference query, respectively. On the other hand, for the approximate nearest neighbor problem (similar arguments can be made for other types of queries), given a set of points P , there is no method to derive a subset S , where $|S| \ll |P|$, such that for any query point $q \in \mathbb{R}^d$, $\text{dist}(q, S) \leq (1 + \varepsilon) \cdot \text{dist}(q, P)$. There are some recent results constructing coresets for approximate nearest neighbor queries with additive approximation [26]. While our current data structures only handle percentile and preference queries (percentile measure and top-k preference measure functions), our work is a first step toward efficiently solving general problems arising from our proposed framework.

- For the CPtile problem in $\mathbb{R}$ we proposed an exact data structure of near linear space and polylogarithmic query, assuming that θ is fixed. A natural question to ask is whether this data structure can be extended to allow θ to be part of the query. Our current data structure (in Appendix B.1) heavily uses the fact that $\theta = [a_\theta, b_\theta]$ is known in the preprocessing phase. One idea to overcome this obstacle is to construct the data structure for different dyadic ranges in $[0, 1]$. Such an idea has been used in [18] for range queries. Given R and θ , we could combine the different results from the precomputed dyadic ranges to have a valid solution for our problem. Currently, it is not clear how the precomputed dyadic ranges can be combined to get an exact solution.

References

- [1] P. Afshani. Improved pointer machine and i/o lower bounds for simplex range reporting and related problems. In *Proceedings of the twenty-eighth annual Symposium on Computational Geometry*, pages 339–346, 2012.
- [2] P. Afshani, P. Cheng, A. Basu Roy, and Z. Wei. On range summary queries. In *50th International Colloquium on Automata, Languages, and Programming (ICALP 2023)*, 2023.
- [3] P. K. Agarwal, S. Har-Peled, and H. Yu. Robust shape fitting via peeling and grating coresets. *Discrete & Computational Geometry*, 39(1):38–58, 2008.
- [4] P. K. Agarwal, X. Hu, S. Sintos, and J. Yang. Dynamic enumeration of similarity joins. In *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, 2021.
- [5] P. K. Agarwal, N. Kumar, S. Sintos, and S. Suri. Efficient algorithms for k-regret minimizing sets. In *16th International Symposium on Experimental Algorithms (SEA 2017)*, 2017.
- [6] A. Asudeh and F. Nargesian. Towards distribution-aware query answering in data markets. *Proceedings of the VLDB Endowment*, 15(11):3137–3144, 2022.
- [7] G. Bagan, A. Durand, and E. Grandjean. On acyclic conjunctive queries and constant delay enumeration. In *International Workshop on Computer Science Logic*, pages 208–222. Springer, 2007.
- [8] L. Behme, S. Galhotra, K. Beedkar, and V. Markl. Fainder: A fast and accurate index for distribution-aware dataset search. *Proceedings of the VLDB Endowment*, 17(11):3269–3282, 2024.
- [9] J. Bethlehem. Selection bias in web surveys. 78(2):161–188.
- [10] S. Bharadwaj, P. Gupta, R. Bhagwan, and S. Guha. Discovering related data at scale. *Proceedings of the VLDB Endowment*, 14(8):1392–1400, 2021.
- [11] A. Bogatu, A. A. Fernandes, N. W. Paton, and N. Konstantinou. Dataset discovery in data lakes. In *2020 IEEE 36th international conference on data engineering (icde)*, pages 709–720. IEEE, 2020.
- [12] D. Brickley, M. Burgess, and N. Noy. Google dataset search: Building a search engine for datasets in an open web ecosystem. In *The world wide web conference*, pages 1365–1375, 2019.
- [13] S. Castelo, R. Rampin, A. Santos, A. Bessa, F. Chirigati, and J. Freire. Auctus: A dataset search engine for data discovery and augmentation. *Proceedings of the VLDB Endowment*, 14(12):2791–2794, 2021.
- [14] R. Castro Fernandez, Z. Abedjan, F. Koko, G. Yuan, S. Madden, and M. Stonebraker. Aurum: A data discovery system. In *34th IEEE International Conference on Data Engineering, ICDE '18*, pages 1001–1012. IEEE.
- [15] T. M. Chan, Q. He, and Y. Nekrich. Further results on colored range searching. In *36th International Symposium on Computational Geometry, SoCG 2020*, pages LIPIcs–SoCG, 2020.
- [16] T. M. Chan and K. Tsakalidis. Dynamic orthogonal range searching on the ram, revisited. *Journal of Computational Geometry*, 9(2):45–66, 2018.
- [17] B. Chazelle. *The discrepancy method: randomness and complexity*. Cambridge University Press, 2000.
- [18] G. Cormode, M. Garofalakis, P. J. Haas, C. Jermaine, et al. Synopses for massive data: Samples, histograms, wavelets, sketches. *Foundations and Trends® in Databases*, 4(1–3):1–294, 2011.

- [19] A. Culotta. Reducing sampling bias in social media data for county health inference. In *Joint Statistical Meetings, JSM '14*. American Statistical Association.
- [20] J. Dastin. Amazon scraps secret AI recruiting tool that showed bias against women. In K. Martin, editor, *Ethics of Data and Analytics*, pages 296–299. Auerbach Publications, 1 edition.
- [21] M. De Berg, M. Van Kreveld, M. Overmars, and O. C. Schwarzkopf. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.
- [22] M. Esmailoghli, J.-A. Quiané-Ruiz, and Z. Abedjan. Mate: multi-attribute table extraction. *Proceedings of the VLDB Endowment*, 15(8):1684–1696, 2022.
- [23] A. Esmailpour, S. Krishnan, and S. Sintos. Range (Rényi) entropy queries and partitioning. *arXiv preprint arXiv:2312.15959*, 2024.
- [24] R. C. Fernandez, Z. Abedjan, F. Koko, G. Yuan, S. Madden, and M. Stonebraker. Aurum: A data discovery system. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, pages 1001–1012. IEEE, 2018.
- [25] S. Galhotra, Y. Gong, and R. C. Fernandez. Metam: Goal-oriented data discovery. In *39th IEEE International Conference on Data Engineering, ICDE '23*, pages 2780–2793. ISSN: 2375-026X.
- [26] J. Gao and C. Long. Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3):1–27, 2024.
- [27] I. Goldstein, T. Kopelowitz, M. Lewenstein, and E. Porat. Conditional lower bounds for space/time tradeoffs. In F. Ellen, A. Kolokolova, and J.-R. Sack, editors, *Algorithms and Data Structures*, pages 421–436, Cham, 2017. Springer International Publishing.
- [28] Y. Gong, S. Galhotra, and R. Castro Fernandez. Nexus: Correlation discovery over collections of spatio-temporal tabular data. *Proceedings of the ACM on Management of Data*, 2(3):1–28, 2024.
- [29] Y. Gong, Z. Zhu, S. Galhotra, and R. C. Fernandez. Ver: View discovery in the wild. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pages 503–516. IEEE, 2023.
- [30] Z. A. Greenacre. The importance of selection bias in internet surveys. 6(3):397–404.
- [31] P. Gupta, R. Janardan, and M. Smid. Further results on generalized intersection searching problems: counting, reporting, and dynamization. *Journal of Algorithms*, 19(2):282–317, 1995.
- [32] S. Har-Peled. *Geometric approximation algorithms*. Number 173. American Mathematical Soc., 2011.
- [33] P. Indyk, S. Mahabadi, M. Mahdian, and V. S. Mirrokni. Composable core-sets for diversity and coverage maximization. In *Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 100–108, 2014.
- [34] H. Kaplan, N. Rubin, M. Sharir, and E. Verbin. Counting colors in boxes. In *SODA*, pages 785–794, 2007.
- [35] C. Koutras, G. Siachamis, A. Ionescu, K. Psarakis, J. Brons, M. Fragkoulis, C. Lofi, A. Bonifati, and A. Katsifodimos. Valentine: Evaluating matching techniques for dataset discovery. In *37th IEEE International Conference on Data Engineering, ICDE '21*, pages 468–479. ISSN: 2375-026X.
- [36] S. Krishnan and S. Sintos. Range entropy queries and partitioning. In *27th International Conference on Database Theory (ICDT 2024)*, 2024.
- [37] N. Kumar and S. Sintos. Faster approximation algorithm for the k-regret minimizing set and related problems. In *2018 Proceedings of the Twentieth Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 62–74. SIAM, 2018.
- [38] S. Lu and Y. Tao. Indexing for keyword search with structured constraints. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 263–275, 2023.
- [39] R. J. Miller, F. Nargesian, E. Zhu, C. Christodoulakis, K. Q. Pu, and P. Andritsos. Making open data transparent: Data discovery on open data. *IEEE Data Eng. Bull.*, 41(2):59–70, 2018.
- [40] M. Mulshine. A major flaw in google’s algorithm allegedly tagged two black people’s faces with the word ‘gorillas’.
- [41] F. Nargesian, A. Asudeh, and H. Jagadish. Tailoring data source distributions for fairness-aware data integration. *Proceedings of the VLDB Endowment*, 14(11):2519–2532, 2021.
- [42] F. Nargesian, K. Pu, B. Ghadiri-Bashardoost, E. Zhu, and R. J. Miller. Data lake organization. 35(1):237–250. Conference Name: IEEE Transactions on Knowledge and Data Engineering.
- [43] F. Nargesian, K. Q. Pu, E. Zhu, B. Ghadiri Bashardoost, and R. J. Miller. Organizing data lakes for navigation. In *ACM International Conference on Management of Data, SIGMOD '20*, pages 1939–1950. ACM Press.
- [44] Y. Nekrich. Efficient range searching for categorical and plain data. *ACM Transactions on Database Systems (TODS)*, 39(1):1–21, 2014.
- [45] N. Noy, M. Burgess, and D. Brickley. Google dataset search: Building a search engine for datasets in an open web ecosystem. In *World Wide Web Conference, WWW '19*, pages 1365–1375.
- [46] P. Ouellette, A. Sciortino, F. Nargesian, B. G. Bashardoost, E. Zhu, K. Q. Pu, and R. J. Miller. RONIN: Data lake exploration. 14(12):2863–2866.
- [47] M. H. Overmars. *The design of dynamic data structures*, volume 156. Springer Science & Business Media, 1983.

- [48] M. H. Overmars and J. van Leeuwen. Worst-case optimal insertion and deletion methods for decomposable searching problems. *Information Processing Letters*, 12(4):168–173, 1981.
- [49] S. Rahul. Approximate range counting revisited. In *33rd International Symposium on Computational Geometry*, 2017.
- [50] S. Rahul and R. Janardan. Algorithms for range-skyline queries. In *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*, pages 526–529, 2012.
- [51] A. Rose. Are face-detection cameras racist?
- [52] T. Townsend. Most engineers are white — and so are the faces they use to train software.
- [53] V. Vapnik and A. Y. Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability and its Applications*, 16(2):264, 1971.
- [54] R. Wang, S. Lu, and Y. Tao. An index for set intersection with post-filtering. *IEEE Trans. on Knowl. and Data Eng.*, 36(7):2862–2876, Nov. 2023.
- [55] A. Yu, P. K. Agarwal, and J. Yang. Processing a large number of continuous preference top-k queries. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*, pages 397–408, 2012.
- [56] H. Zhang, A. Santos, and J. Freire. DSDD: Domain-specific dataset discovery on the web. In *30th ACM International Conference on Information & Knowledge Management, CIKM '21*, pages 2527–2536. ACM Press. event-place:.
- [57] Y. Zhang and Z. G. Ives. Finding related tables in data lakes for interactive data science. In *ACM International Conference on Management of Data, SIGMOD '20*, pages 1951–1966. ACM Press.
- [58] H. Zhao, S. Deep, and P. Koutris. Space-time tradeoffs for conjunctive queries with access patterns. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS '23*, page 59–68, New York, NY, USA, 2023. Association for Computing Machinery.
- [59] J. Zhu, S. Galhotra, N. Sabri, and B. Salimi. Consistent range approximation for fair predictive modeling. 16(11):2925–2938.

A Missing details from Section 2

PROOF OF LEMMA 2.1. Since $\text{ERR}_{S_X}(\mathcal{F}_{\square}^d) = \delta$, we have that for every rectangle $R \in \mathbb{R}^d$, $|\mathcal{M}_R(X) - \mathcal{M}_R(S_X)| \leq \delta \Leftrightarrow -\delta + \mathcal{M}_R(S_X) \leq \mathcal{M}_R(X) \leq \delta + \mathcal{M}_R(S_X) \Leftrightarrow -\delta + \frac{|R \cap S_X|}{|S_X|} \leq \frac{|R \cap X|}{|X|} \leq \delta + \frac{|R \cap S_X|}{|S_X|}$. Since S is a set of $O(\varepsilon^{-2} \log \varphi^{-1})$ random samples from S_X , from [17], we have that S is an ε -sample for S_X with probability at least $1 - \varphi^{-1}$. Hence, by definition, $-\varepsilon + \frac{|R \cap S|}{|S|} \leq \frac{|R \cap S_X|}{|S_X|} \leq \varepsilon + \frac{|R \cap S|}{|S|}$. By combining the derived inequalities, $-\delta - \varepsilon + \frac{|R \cap S|}{|S|} \leq \frac{|R \cap X|}{|X|} \leq \delta + \varepsilon + \frac{|R \cap S|}{|S|} \Leftrightarrow \left| \frac{|R \cap X|}{|X|} - \frac{|R \cap S|}{|S|} \right| \leq \delta + \varepsilon$. The result follows. $\square$

B Missing details from Section 3

PROOF OF LEMMA 3.3. Throughout this proof, we use the term *total size* of a collection of sets to denote the sum of the sizes of the sets in the collection.

We show that if a data structure $\mathbf{D}$ exists for the uniform set intersection problem with space near-linear on total size and query time near-linear on output size, we can break the strong set intersection conjecture. Intuitively, we show that if $\mathbf{D}$ exists, we can use it as a black box to efficiently answer general set intersection queries.

Let $S = \{S_1, \dots, S_g\}$, be an arbitrary collection of sets without any restrictions over a universe $\mathcal{U}$, with total size $W = \sum_{S_i \in S} |S_i|$. We build a data structure $\mathbf{D}'$ to answer intersection queries over S as follows. For any element $u \in \mathcal{U}$, let $c_u = |\{S_i \in S : u \in S_i\}|$ be the number of different sets that u belongs to. First, we partition $\mathcal{U}$ into three subsets, the *light* elements $\mathcal{U}^l = \{u \in \mathcal{U} : c_u < W^{\frac{5}{12}}\}$, the *medium* elements $\mathcal{U}^m = \{u \in \mathcal{U} : W^{\frac{5}{12}} \leq c_u < W^{\frac{5}{6}}\}$, and the *heavy* elements $\mathcal{U}^h = \{u \in \mathcal{U} : W^{\frac{5}{6}} \leq c_u\}$. Next, for every pair $i, j \in [g]$, such that $S_i \cap S_j \cap \mathcal{U}^l \neq \emptyset$ we store the list $L_{i,j} = S_i \cap S_j \cap \mathcal{U}^l$. We place all computed lists in a simple lookup table with respect to the indexes i, j . For each $S_i \in S$, we define $T_i = S_i \cap \mathcal{U}^m$. Let T be an initially empty collection of sets. We add T_i to T for every $S_i \in S$. Our goal is to make T uniform and to do so, we add dummy sets to T . More formally, for any element $u \in \mathcal{U}^m$, let $c'_u = |\{T_i \in T : u \in T_i\}|$ be the number of different sets in T that u belongs to. We add $W^{\frac{5}{6}} - c'_u$ identical sets $\{u\}$ to T for all $u \in \mathcal{U}^m$ to make T uniform. Let M be the total size of T after these operations. We build the data structure $\mathbf{D}$ over T . Since T is uniform, by the assumption, $\mathbf{D}$ has size $\tilde{O}(M)$ and answers set intersection queries over T in $\tilde{O}(1 + \text{OUT})$ time. At this stage, we are done with the preprocessing of $\mathbf{D}'$ and we are ready to answer set intersection queries over S .

When an intersection query for a pair of sets S_i, S_j comes, we first look for the pair i, j in the lookup table to see whether S_i and S_j intersect on a light value. If the answer is positive we report the elements in $L_{i,j}$. Next, we run the intersection query for T_i and T_j on $\mathbf{D}$ and report all the medium elements in $S_i \cap S_j$ and continue. Finally, we go through each heavy element $u \in \mathcal{U}^h$ one by one and check whether both S_i and S_j contain u and report it if the answer is positive.

It is easy to verify that by the described procedure, we correctly answer the queries since we report the light values in $S_i \cap S_j$ using the preprocessed lookup table, the medium values in $S_i \cap S_j$ invoking $\mathbf{D}$, and the heavy values in $S_i \cap S_j$ considering every heavy value one by one.

Now, we to analyze the space and query time of $\mathbf{D}'$ as described above. For the query phase, we check the lookup table in $\tilde{O}(1)$ time and report the light elements in $S_i \cap S_j$ in $\tilde{O}(\text{OUT})$ time. We invoke $\mathbf{D}$ once and spend $\tilde{O}(\text{OUT})$ time to report the heavy elements in $S_i \cap S_j$. We check all heavy elements in $\tilde{O}(|\mathcal{U}^h|)$ time. We have $\sum_{u \in \mathcal{U}} c_u = \sum_{S_i \in S} |S_i| = W$, and for each heavy value $u \in \mathcal{U}^h$, we know $c_u \geq W^{\frac{5}{6}}$ by definition. Thus, we have $|\mathcal{U}^h| \leq \frac{W}{W^{\frac{5}{6}}} = W^{\frac{1}{6}}$. Therefore, $\mathbf{D}'$ answers the intersection queries in $\tilde{O}(W^{\frac{1}{6}} + \text{OUT})$ time.

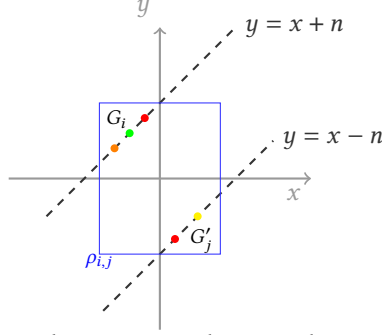


Fig. 4. An example of the lower bound construction. The rectangle $\rho_{i,j}$ only contains the points from G_i and G'_j . The points from same the same point sets are shown with the same color. Since red appears twice in $\rho_{i,j}$, the sets S_i and S_j intersect.

Next, we bound the space of $\mathbf{D}'$. For each medium element $u \in \mathcal{U}^m$, we have $c_u \geq W^{\frac{5}{12}}$ by definition and hence we have $|\mathcal{U}^m| \leq \frac{W}{W^{\frac{5}{12}}} = W^{\frac{7}{12}}$. By the construction of T , we know that each $u \in \mathcal{U}^m$ belongs to exactly $W^{\frac{5}{6}}$ different sets in T , so the total size of T is equal to $|\mathcal{U}^m|W^{\frac{5}{6}} \leq W^{\frac{7}{12}}W^{\frac{5}{6}} = W^{\frac{17}{12}}$. So, by the assumption, $\mathbf{D}$ uses $\tilde{O}(W^{\frac{17}{12}})$ space. To bound the size of the lookup table, we need to bound the number of the pairs of sets that intersect on a light element. We have $\sum_{S_i \in S} |S_i| = W$ and hence there are at most W pairs of set-elements like (S_i, u) such that $S_i \in S$, $u \in S_i$, and $u \in \mathcal{U}^l$. By definition, we know that for each light element u , it holds that $c_u < W^{\frac{5}{12}}$ and hence each of these set-element pairs can be matched to at most $W^{\frac{5}{12}}$ other set-element pair on the same element to create a pair of sets that intersect on a light value. Thus, there are at most $W^{\frac{5}{12}}W = W^{\frac{17}{12}}$ different set-element, set-element pairs where the element is a single light value, so $\sum_{\ell_1, \ell_2 \in [g]} |L_{\ell_1, \ell_2}| = \tilde{O}(W^{\frac{17}{12}})$ and the size of the lookup table is $\tilde{O}(W^{\frac{17}{12}})$.

Overall, $\mathbf{D}'$ uses $\tilde{O}(W + W^{\frac{17}{12}} + W^{\frac{17}{12}}) = \tilde{O}(W^{\frac{17}{12}})$ space to answer queries in $\tilde{O}(W^{\frac{1}{6}} + \text{OUT})$ time over an input S with total size of W . According to Conjecture 3.2, any data structure that answers intersection queries in $\tilde{O}(W^{\frac{1}{6}} + \text{OUT})$, needs space of $\tilde{\Omega}\left(\frac{W^2}{W^{\frac{5}{6}}}\right) = \tilde{\Omega}(W^{\frac{5}{3}})$. This implies that $\mathbf{D}$ does not exist since $W^{\frac{5}{3}} > W^{\frac{17}{12}}$, and the result follows. $\square$

B.1 Reduction from Set Intersection to CPtile

We show that having an efficient data structure for answering the CPtile query contradicts Lemma 3.3 and hence the existence of efficient data structures for the CPtile problem is unlikely.

Let $S_1, \dots, S_g$ be a uniform collection of sets. Let $M = \sum_{i \in [g]} |S_i|$ and let $s_{i,k}$ be the value of the k -th item in S_i (we consider any arbitrary order of the items in each S_i). Without loss of generality, we assume that $\mathcal{U} = \cup_{i \in [g]} S_i = \{1, \dots, q\}$, for some integer $q \leq M$.

Using the idea from [50], we design an instance $\mathcal{P} = \{P_1, \dots, P_q\}$ of the CPtile problem in $\mathbb{R}^2$ based on these sets. All points lie on two parallel lines $L = x + M$, and $L' = x - M$. Let $m_0 = 0$ and $m_i = m_{i-1} + |S_i|$ for $i = 1, \dots, g$. For each $s_{i,k}$ we create two points, $p_{i,k} = (-(k + m_{i-1}), -(k + m_{i-1}) + M)$ on L , and $p'_{i,k} = ((k + m_{i-1}), k + m_{i-1} - M)$ on L' and we add both points $p_{i,k}, p'_{i,k}$ to the point set $P_{s_{i,k}}$. Let G_i be the set of points corresponding to S_i that lie on L , and G'_i be the set of points corresponding to S_i that lie on L' . Let $H = \cup_{i \in [g]} P_i$ denote the set of all the points we created.

Given any pair of indices $i, j \in [g]$, in the set intersection problem, we want to report $S_i \cap S_j$. Geometrically, it is known [50] that we can find a rectangle $\rho_{i,j}$ in $O(1)$ time such that $\rho_{i,j} \cap H =$

$G_i \cup G'_j$ (see Figure 4). Hence, it is easy to check that for any element $u \in \mathcal{U}$, we have $u \in S_i \cap S_j$, if and only if $|P_u \cap \rho_{i,j}| = 2$. Moreover, since we started from a uniform collection of sets, all the point sets contain the same number of points, i.e., for any pair of point sets $P_i, P_j \in \mathcal{P}$, we have $|P_i| = |P_j| = t$, for some number t . To answer the intersection queries, we build a data structure $\mathbf{D}$ for answering CPtile query over $\mathcal{P}$. For an arbitrary pair of sets S_i, S_j , we first find $\rho_{i,j}$ and then query $\mathbf{D}$ with rectangle $\rho_{i,j}$ and interval $[\frac{1.5}{t}, 1]$. Note that $\mathbf{D}$ reports an index r , if and only if $|P_r \cap \rho_{i,j}| > 1$ (so $|P_r \cap \rho_{i,j}| = 2$), and hence, $\mathbf{D}$ reports $S_i \cap S_j$.

For an input of size x , let $\mathcal{S}(x)$ be the space used by a data structure for the CPtile and $Q(x) + \text{OUT}$ be its query time, where OUT is the output size. By the above construction, we can answer uniform set intersection queries in time $O(1) + Q(M) + \text{OUT}$ and space $\mathcal{S}(M) + M$, using $\mathbf{D}$. Along with Lemma 3.3, we conclude with Theorem 3.4.

B.2 Reduction from Halfspace reporting to CPref

We show a reduction from the halfspace reporting problem to CPref problem. Let U be a set of n points in $\mathbb{R}^d$. Without loss of generality, we can assume that they all lie in the first orthant (otherwise we can apply a transformation to move all the points to the first orthant without changing the definition of the problem). Furthermore, without loss of generality, we can assume that U lies in the unit ball. If not, then we can apply a scaling transformation to map U to the set $\bar{U}$ in the unit ball. It is known that for any halfspace H we can construct a halfspace $\bar{H}$ in $O(1)$ time, such that $u \in U \cap H$ if and only if $\bar{u} \in \bar{U} \cap \bar{H}$. Let $\mathcal{P} = \{P_1, \dots, P_n\}$ be a repository such that for every $i \in [n]$, the dataset P_i contains only the point $u_i \in U$. Let $\mathbf{D}$ be a data structure for the CPref problem over $\mathcal{P}$. We also apply the following affine transformation to U . Let $x \in \mathbb{R}^d$ be a point such that all coordinates of the points in U are smaller than x 's coordinates. We apply a rotation transformation to U creating U' such that x becomes the origin and U' lies in the first orthant. Such a rotation is computed in $O(n)$ time. Let $\mathcal{P}' = \{P'_1, \dots, P'_n\}$ be a repository such that for every $i \in [n]$, the dataset P'_i contains only the point $u'_i \in U'$. Let $\mathbf{D}'$ be a data structure for the CPref problem over $\mathcal{P}'$.

Now assume that a query halfspace H is given for the halfspace reporting problem. We check in $O(1)$ time whether H intersects the first orthant. If not, we return $\emptyset$. Otherwise, we proceed as follows. We check in $O(1)$ time whether the origin belongs in H . If not, then: Let h be the halfplane that defines H . We compute in $O(1)$ time a unit vector $\vec{v}$ perpendicular to h . We also compute in $O(1)$ time the line L that supports $\vec{v}$ and passes through the origin. Let a_θ be the distance from the origin to the intersection point of L and h . Let $\theta = [a_\theta, 1]$ and $k = 1$. We query $\mathbf{D}$ on predicate $\text{PRED}_{\mathcal{M}_{\vec{v},k},\theta}$. On the other hand, if the origin belongs in H , then we apply the same rotation we had from U to U' , to the halfspace H to get H' , in $O(1)$ time. We repeat the same procedure as before to compute $\vec{v}'$ and $a_{\theta'}$. Let $\theta' = [a_{\theta'}, 1]$ and $k' = 1$. We query $\mathbf{D}'$ on predicate $\text{PRED}_{\mathcal{M}_{\vec{v}',k'},\theta'}$.

When H does not contain the origin, it is easy to see that $u_i \in H$ if and only if $\text{PRED}_{\mathcal{M}_{\vec{v},k},\theta}(P_i) = \text{True}$. Similarly, if H contains the origin, $u_i \in H$ if and only if $u'_i \in H'$. And $u'_i \in H'$ if and only if $\text{PRED}_{\mathcal{M}_{\vec{v}',k'},\theta'}(P'_i) = \text{True}$.

Let $\mathcal{S}(n)$ be the space of the data structure for the CPref problem on n datasets and $Q(n) + \text{OUT}$ be its query time, where OUT is the output size. Using $\mathbf{D}, \mathbf{D}'$, we can solve the halfspace reporting problem with a data structure of $2\mathcal{S}(n)$ space and $O(1) + Q(n) + \text{OUT}$ query time.

C Missing details from Section 4

C.1 Exact data structure for CPtile in $\mathbb{R}^1$

In the CPtile problem, we have full access to P_i for every $i \in [N]$. We assume that we know the range-predicate θ upfront, i.e., $\theta = [a_\theta, b_\theta]$ is not part of the query. This is only a requirement in

this subsection. In all the other general approximate data structure θ is part of the query. The data structure works for points in $\mathbb{R}^1$.

Main idea. The main issue to answer a CPtile query is that many points from the same dataset might lie inside the query rectangle (interval in $\mathbb{R}^1$) R . Instead, for every dataset P_i we map all its points to a new dataset in $\mathbb{R}^4$, such that given R , there exists an orthant R' in $\mathbb{R}^4$ satisfying the following strong property: For every dataset there will be at most one point in R' , and for every dataset P_i that has a point in R' we guarantee that i must be reported to answer the CPtile query correctly.

Data structure. Let $P_i = \{p_1^{(i)}, \dots, p_{n_i}^{(i)}\}$ for every $P_i \in \mathcal{P}$. Without loss of generality, we also assume that $p_{h_1}^{(i)} \neq p_{h_2}^{(i)}$ for every $i \in [N]$ and $h_1 \neq h_2 \in [n_i]$. For every $i \in [N]$, we sort the points in P_i in ascending order. Without loss of generality assume $p_1^{(i)} < \dots < p_{n_i}^{(i)}$. For every point $p_j^{(i)} \in P_i$, let q_j be the point in P_i such that $|(q_j, p_j^{(i)}) \cap P_i| = b_\theta \cdot n_i$, let r_j be the point in P_i such that $|(r_j, p_j^{(i)}) \cap P_i| = a_\theta \cdot n_i$, and let $s_j = p_{j+1}^{(i)}$. We straightforwardly handle the corner cases, for example if s_j is not defined because $p_{j+1}^{(i)}$ is the rightmost point in P_i , we can assume that $s_j \rightarrow \infty$. Let $Q_i = \{(q_j, r_j, p_j^{(i)}, s_j) \mid p_j^{(i)}\}$ be the set of n_i four dimensional points. Let $Q = \bigcup_{i \in [N]} Q_i$. We construct a (static) range tree $\mathcal{T}$ for range reporting queries on Q .

Query procedure. Let $R = [R^-, R^+]$ be the query interval in $\mathbb{R}^1$. We define the orthant $R' = (-\infty, R^-] \times [R^-, \infty) \times (-\infty, R^+] \times [R^+, \infty)$ in $\mathbb{R}^4$ and we run a reporting query on $\mathcal{T}$. Each time the query procedure finds a point that belongs in Q_i , we report the index i .

Correctness.

LEMMA C.1. *The query procedure does not report duplicates*

PROOF. For every $i \in [N]$, we show that there is at most one point $p_j^{(i)}$ such that $(q_j, r_j, p_j^{(i)}, s_j) \in R' \cap Q_i$. We observe that only the point in P_i with the largest coordinate in R satisfies the last two linear constraints of the orthant. If $p_j^{(i)}$ is the point with the largest coordinate in R among the points in P_i then $p_j^{(i)} \leq R^+$ and $s_j > R^+$. For any $p_h^{(i)}$ with $h < j$ it does not hold that $s_j \geq R^+$, while for any $p_h^{(i)}$ with $h > j$ it does not hold that $p_h^{(i)} \leq R^+$. $\square$

Let $\Pi = \text{PRED}_{\mathcal{M}_{R,\theta}}$ be the logical expression with the range-predicate θ . Recall that $\mathbf{q}_\Pi(\mathcal{P})$ denotes the correct set of indexes we should report.

LEMMA C.2. *The query procedure returns $\mathbf{q}_\Pi(\mathcal{P})$.*

PROOF. Let J be the set of indexes reported by the query procedure. Let $i \in \mathbf{q}_\Pi(\mathcal{P})$. Let $p_j^{(i)}$ be the points in $P_i \cap R$ with the largest coordinate. From Lemma C.1, the last two linear constraints of R' are satisfied. Since $i \in \mathbf{q}_\Pi(\mathcal{P})$ it must be the case that $a_\theta \leq \frac{|P_i \cap R|}{|P_i|} \leq b_\theta \Leftrightarrow a_\theta \cdot n_i \leq |P_i \cap R| \leq b_\theta \cdot n_i \Leftrightarrow q_j \leq R^-$ and $r_j \geq R^-$. Hence, $(q_j, r_j, p_j^{(i)}, s_j) \in R'$ and the index i will be reported by the query procedure. Equivalently, if $i \notin \mathbf{q}_\Pi(\mathcal{P})$, then $(q_j, r_j, p_j^{(i)}, s_j) \notin R'$ and the index i will not be reported by the query procedure. $\square$

Analysis.

LEMMA C.3. *The data structure has $O(N \log^3(N))$ space and it can be constructed in $O(N \log^3(N))$ time.*

PROOF. For every point $p_j^{(i)}$ in P_i we add one point in Q_i . The result follows from the complexity of constructing and storing a static range tree on N 4-dimensional points. $\square$

LEMMA C.4. *The query time is $O(\log^3(N \cdot n) + |J|)$.*

PROOF. It follows from the proof of Lemma C.1 and from the query procedure of a 4-dimensional range tree. $\square$

THEOREM C.5. *Let $\mathcal{P} = \{P_1, \dots, P_N\}$ be the input to the CPtile problem, such that $N = \sum_{i \in [N]} |P_i|$, and $P_i \subset \mathbb{R}$ for every $i \in [N]$. Let θ be an interval in $\mathbb{R}$. A data structure of size $O(N \log^3(N))$ can be constructed in $O(N \log^3(N))$ time, such that given a query $\Pi = \text{PRED}_{M_R, \theta}$, it returns the set of indexes $q_\Pi(\mathcal{P})$ in $O(\log^3(N) + |q_\Pi(\mathcal{P})|)$ query time.*

Remark 1. The data structure in Theorem C.5 can be made dynamic under insertion or deletion of datasets or points in the datasets. Instead of using a static range tree, we would use a dynamic range tree as discussed in Section 2. If a new dataset P with n points is inserted (resp. deleted) in (resp. from) $\mathcal{P}$, then the data structure is updated in $O(n \log^4(N + n))$ time. Furthermore, if a point is inserted/deleted from a current dataset, then the data structure can be updated in $O(\log^4 N)$ time.

Remark 2. The range tree supports delay guarantees. Since we do not report duplicates, we can report $q_\Pi(\mathcal{P})$ with delay guarantee $O(\log^3 N)$.

C.2 Missing proofs from Subsection 4.2

PROOF OF LEMMA 4.3. For simplicity, we assume that we can get one sample from S_{P_i} in $O(1)$ time, for every $i \in [N]$. We have $|S_i| = O(\varepsilon^{-2} \log(N\varphi^{-1}))$, so there are $O(\varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$ combinatorially different hyper-rectangles defined by S , i.e., $|\mathcal{R}_i| = O(\varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$. Hence, $|Q| = O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$ and it is constructed in $O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$ time. Using an additional static range tree on S_i for counting queries, we also compute all the values in W in $O(N \cdot \varepsilon^{-4d} \log^{2d-1}(N\varphi^{-1}) \log^d(\varepsilon^{-1} \log(N\varphi^{-1})))$ time. After computing Q and W , the range tree $\mathcal{T}$ is constructed in $O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{2d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time and it has $O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{2d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ space. Overall, our data structure is constructed in $O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{2d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time and it has $O(N \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{2d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ space.

Next, we focus on the query time. Every query in $\mathcal{T}$ takes $O(\log^{2d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time. Every update operation takes $O(\log^{2d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time. Each time we report an index j , we delete all points in Q_j . From Lemma 4.3, we have $|Q_j| = O(\varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$. The overall query time is $O(\log^{2d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})) + |J| \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{2d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$. $\square$

C.3 Missing proofs from Subsection 4.3

PROOF OF LEMMA 4.9. Notice that every time we report an index j , we do not remove all points in Q_j . Instead, we only remove the points $\{q_{(\rho, \bar{\rho})} \mid \rho \in \mathcal{R}_j\}$. From Lemma 4.5, any time that our query procedure finds $q_{(\rho, \bar{\rho})}$, ρ must be the largest hyper-rectangle inside R . Hence, by removing all points $\{q_{(\rho, \bar{\rho})} \mid \rho \in \mathcal{R}_j\}$, the index j will not be reported again. $\square$

PROOF OF LEMMA 4.10. We assume that we get one sample from S_{P_i} in $O(1)$ time, for every $i \in [N]$. We have $|S_i| \leq |\bar{S}_i| = O(\varepsilon^{-2} \log(N\varphi^{-1}))$. So

$$|\mathcal{R}_i| = O(\varepsilon^{-4d} \log^{2d}(N\varphi^{-1})), \quad \text{and} \quad |\hat{\mathcal{R}}_i| = O(\varepsilon^{-8d} \log^{4d}(N\varphi^{-1})).$$

Hence, $|Q| = O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}))$ and it is constructed in

$$O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^{2d}(\varepsilon^{-1} \log(N\varphi^{-1})))$$

time, using a range tree to check whether there exists ρ' such that $\rho \subseteq \rho' \subseteq \hat{\rho}$. Using an additional static range tree on S_i for counting queries, we also compute W in

$$O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^d(\varepsilon^{-1} \log(N\varphi^{-1})))$$

time. After computing Q and W , the range tree $\mathcal{T}$ is constructed in $O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^{4d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time and it has $O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^{4d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ space. Overall, our data structure is constructed in $O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^{4d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time and it has $O(N \cdot \varepsilon^{-8d} \log^{4d}(N\varphi^{-1}) \log^{4d}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ space.

Next, we focus on the query time. Every query in $\mathcal{T}$ takes $O(\log^{4d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time. Every update operation takes $O(\log^{4d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time. Each time we report an index j , we delete $O(\varepsilon^{-4d} \log^{2d}(N\varphi^{-1}))$ points from Q_j . The overall query time is $O(\log^{4d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})) + |J| \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{4d+1}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$. $\square$

C.4 Approximate data structure for the Ptile problem with a logical expression

So far, we assumed that the (query) logical expression Π contains only one predicate, $\text{PRED}_{M_R, \theta}$. In this subsection, we extend our data structure to work for any constant number m of predicates. Handling disjunction of predicates is straightforward using the designed data structure. For example, if the query consists of rectangles $R^{(1)}, \dots, R^{(m)}$ with intervals $\theta^{(1)}, \dots, \theta^{(m)}$ then by keeping track of the indexes we report (so that we do not report duplicates), we can handle every rectangle independently. We might encounter an index i at most $O(m) = O(1)$ times so all our guarantees from the previous section hold. On the other hand, it is not clear how to handle conjunction of predicates.

Main idea. In this section, we describe a data structure that handles m conjunctions of predicates. The main difference from the previous section is that we map all possible m pairs of rectangles $(\rho, \hat{\rho})$ in $4md$ dimensions instead of $4d$ dimensions that we had in the previous section. Given a query rectangle R and m predicates we define an orthant R' in $4md$ dimensions. The construction of the data structure is similar as in Subsection 4.3.

We only describe the data structure for general range-predicates. If we consider threshold-predicates the exponential complexities on d are improved.

Data structure. The first steps of constructing the data structure are identical with the data structure in the previous subsection. Without loss of generality assume that all datasets and query rectangles lie in a bounded box $\mathcal{B}$. For every $i \in [N]$, we get an $(\varepsilon + \delta)$ -sample $S_i \subset \mathbb{R}^d$ by sampling $O(\varepsilon^{-2} \log(N\varphi^{-1}))$ points from S_{p_i} uniformly at random. Let $\tilde{S}_i$ be the projection of S_i onto the $2 \cdot d$ boundaries of $\mathcal{R}$. For every $i \in [N]$, we construct the set $\mathcal{R}_i$ that contains all combinatorially different hyper-rectangles defined by $S_i \cup \tilde{S}_i$. Every hyper-rectangle $\rho \in \mathcal{R}_i$ is defined by their two opposite corners $\rho^-, \rho^+ \in \mathbb{R}^d$ such that $\rho_h^- \leq \rho_h^+$, for every $h \in [d]$, where ρ_h^-, ρ_h^+ are the h -coordinates of ρ^- and ρ^+ , respectively. We construct all pairs of rectangles $\hat{\mathcal{R}}_i = \{(\rho, \hat{\rho}) \mid \rho \in \mathcal{R}_i, \hat{\rho} \in \mathcal{R}_i, \rho \subseteq \hat{\rho}, \nexists \rho' \in \mathcal{R}_i : \rho \subset \rho' \subset \hat{\rho}\}$. Let $\tilde{\mathcal{R}}_i$ contain all subsets of m pairs of rectangles in $\hat{\mathcal{R}}_i$. For every $\sigma = \{(\rho^{(1)}, \hat{\rho}^{(1)}), \dots, (\rho^{(m)}, \hat{\rho}^{(m)})\} \in \tilde{\mathcal{R}}_i$, let

$$q_\sigma = (\rho_1^{(1)-}, \dots, \rho_d^{(1)-}, \hat{\rho}_1^{(1)-}, \dots, \hat{\rho}_d^{(1)-}, \rho_1^{(1)+}, \dots, \rho_d^{(1)+}, \hat{\rho}_1^{(1)+}, \dots, \hat{\rho}_d^{(1)+}, \dots, \rho_1^{(m)-}, \dots, \rho_d^{(m)-}, \hat{\rho}_1^{(m)-}, \dots, \hat{\rho}_d^{(m)-}, \rho_1^{(m)+}, \dots, \rho_d^{(m)+}, \hat{\rho}_1^{(m)+}, \dots, \hat{\rho}_d^{(m)+})$$

be the point in $\mathbb{R}^{4md}$ defined by merging the opposite corners of $(\rho^{(r)}, \hat{\rho}^{(r)}) \in \sigma$, for $r \in [m]$. For every point q_σ , we assign m weights, $w_{q_\sigma} = (M_{\rho^{(1)}}(S_i), \dots, M_{\rho^{(m)}}(S_i)) = (\frac{|\rho^{(1)} \cap S_i|}{|S_i|}, \dots, \frac{|\rho^{(m)} \cap S_i|}{|S_i|})$. Let $Q_i = \{q_\sigma \mid \sigma \in \tilde{\mathcal{R}}_i\}$ and $W_i = \{w_{q_\sigma} \mid q_\sigma \in Q_i\}$. Finally, we define $Q = \bigcup_{i \in [N]} Q_i$ and

$W = \bigcup_{i \in [N]} W_i$. We construct a dynamic range tree $\mathcal{T}$ over the (weighted) points Q with their corresponding m -dimensional weights in W .

Query procedure. We are given m query rectangles $\{R^{(1)}, \dots, R^{(m)}\}$ in $\mathbb{R}^d$ and m intervals $\theta^{(1)} = [a_{\theta^{(1)}}, b_{\theta^{(1)}}], \dots, \theta^{(m)} = [a_{\theta^{(m)}}, b_{\theta^{(m)}}]$. Let R^-, R^+ be the two opposite corners of a rectangle R . We define the orthant $R' = [R_1^{(1)-}, \infty) \times \dots \times [R_d^{(1)-}, \infty) \times (-\infty, R_1^{(1)-}) \times \dots \times (-\infty, R_d^{(1)-}) \times (-\infty, R_1^{(1)+}] \times \dots \times (-\infty, R_d^{(1)+}] \times (R_1^{(1)+}, \infty) \times \dots \times (R_d^{(1)+}, \infty) \times \dots \times R_1^{(m)-}, \infty) \times \dots \times [R_d^{(m)-}, \infty) \times (-\infty, R_1^{(m)-}) \times \dots \times (-\infty, R_d^{(m)-}) \times (-\infty, R_1^{(m)+}) \times \dots \times (-\infty, R_d^{(m)+}) \times (R_1^{(m)+}, \infty) \times \dots \times (R_d^{(m)+}, \infty)$ in $\mathbb{R}^{4md}$ and the weight query rectangle $I' = [a_{\theta^{(1)}} - \varepsilon - \delta, b_{\theta^{(1)}} + \varepsilon + \delta] \times \dots \times [a_{\theta^{(m)}} - \varepsilon - \delta, b_{\theta^{(m)}} + \varepsilon + \delta]$. We repeat the following until no index is returned: Using $R' \times I'$ we query the range tree $\mathcal{T}$. Let u be the first (non-empty) canonical node it returns and let q_σ be any arbitrary point stored in this node. Let j be the index of the point set P_j that q_σ was constructed for. We report j , and we delete from $\mathcal{T}$ all points $\{q_\sigma \mid \rho^{(1)} \in \mathcal{R}_j\}$. We continue with the next iteration. If there is no such canonical node or we do not find any q_σ we stop the execution. Let J be the indexes we reported. In the end of the process, we re-insert in $\mathcal{T}$ all points we removed from Q_j for every index $j \in J$.

Correctness. The correctness of the query procedure follows from simple modifications of Lemmas 4.5, 4.6, 4.7, and 4.8. Let $\Pi = \text{PRED}_{\mathcal{M}_{R^{(1)}, \theta^{(1)}}} \wedge \dots \wedge \text{PRED}_{\mathcal{M}_{R^{(m)}, \theta^{(m)}}}$ be the logical expression consisting of the conjunction of m range-predicates. It holds that $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$, with probability at least $1 - \varphi$. Indeed, by extending the proof of Lemma 4.7, we have that: For an index $i \in \mathbf{q}_\Pi(\mathcal{P})$, let $\rho^{(i)}$ be the largest rectangle in $\mathcal{R}_i$ such that $\rho^{(i)} \subseteq R^{(i)}$. Let q_σ be the point in Q_i constructed by $\rho^{(1)}, \dots, \rho^{(m)}$. From Lemma 4.7, it is straightforward to see that $q_\sigma \in R' \cap Q$ and $w_{q_\sigma} \in [a_{\theta^{(1)}} - \varepsilon - \delta, b_{\theta^{(1)}} + \varepsilon + \delta] \times \dots \times [a_{\theta^{(m)}} - \varepsilon - \delta, b_{\theta^{(m)}} + \varepsilon + \delta]$ and the result follows. For every $j \in J$, let $q_\sigma \in Q_j$ be the point found in R' at the moment we reported j . From Lemma 4.5, we know that $\rho^{(\ell)}$ is the largest rectangle inside $R^{(\ell)}$ so the result follows by the definition of ε -sample following the proof of Lemma 4.8. Finally, by removing $\{q_\sigma \mid \rho^{(1)} \in \mathcal{R}_j\}$ we make sure that we will not report the index j again, because any time that our query procedure finds q_σ , the rectangle $\rho^{(\ell)}$ must be the largest rectangle inside $R^{(\ell)}$, for every $\ell \in [m]$.

Analysis. The next two lemmas follow straightforwardly by extending the proof of Lemmas 4.10.

LEMMA C.6. *The constructed data structure has*

$$O(N \cdot \varepsilon^{-8md} \log^{4md} (N\varphi^{-1}) \log^{4md-1+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$$

space and it can be constructed in

$$O(N \cdot \varepsilon^{-8md} \log^{4md} (N\varphi^{-1}) \log^{4md-1+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$$

time.

LEMMA C.7. *The query procedure runs in*

$$O(\log^{4md+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})) + |J| \cdot \varepsilon^{-4d} \log^{2d} (N\varphi^{-1}) \log^{4md+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$$

time.

We set $\varepsilon \leftarrow \varepsilon/2$ and we get the final result.

THEOREM C.8. *Let $\{\mathcal{S}_{P_1}, \dots, \mathcal{S}_{P_N}\}$ be the input to the Ptile problem, such that $\mathcal{S}_{P_i}$ is a synopsis of a dataset $P_i \subset \mathbb{R}^d$, where $d \geq 1$ is a constant, with $\text{ERR}_{\mathcal{S}_{P_i}}(\mathcal{F}_\square^d) \leq \delta$ for $\delta \in [0, 1)$, for every $i \in [N]$. Let $\varepsilon, \varphi \in (0, 1)$ and $m > 1$ be three parameters. A data structure of size $O(N \cdot \varepsilon^{-8md} \log^{4md} (N\varphi^{-1}) \log^{4md-1+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ can be constructed in $O(N \cdot \varepsilon^{-8md} \log^{4md} (N\varphi^{-1}) \log^{4md-1+m} (N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$ time, such that given m query rectangles $R^{(1)}, \dots, R^{(m)}$ and m query intervals $\theta^{(1)}, \dots, \theta^{(m)}$, it returns a set of indexes J such that $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$, where $\Pi =$*

$PRED_{\mathcal{M}_{R(1)}, \theta(1)} \wedge \dots \wedge PRED_{\mathcal{M}_{R(m)}, \theta(m)}$, and for every $j \in J$, $a_{\theta(\ell)} - \varepsilon - 2\delta \leq \mathcal{M}_{R(\ell)}(P_j) \leq b_{\theta(\ell)} + \varepsilon + 2\delta$, for $\ell \in [m]$, with probability at least $1 - \varphi$. The query time is $O(\log^{4md+m}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})) + |J| \cdot \varepsilon^{-4d} \log^{2d}(N\varphi^{-1}) \log^{4md+m}(N \cdot \varepsilon^{-1} \log(N\varphi^{-1})))$.

While in Theorem C.8 we only mention conjunction of predicates, the same result holds if we have any disjunction/conjunction of m predicates. If we assume that ε is an arbitrarily small constant, the success probability is high (at least $1 - \frac{1}{N}$, and m is a constant, then the data structure has size $O(N \cdot \log^{8md-1+m} N) = \tilde{O}(N)$, can be constructed in $O(N \cdot \log^{8md-1+m} N) = \tilde{O}(N)$ time, and it has $O(\log^{4md+m} N + |J| \cdot \log^{4md+2d+m} N) = \tilde{O}(1 + |J|)$ query time.

Remark 1. The data structure in Theorem C.8 can be made dynamic under insertion or deletion of synopses. Using the update procedure of the dynamic range tree, if a synopsis $\mathcal{S}_P$ is inserted/deleted we can update our data structure in $O(\varepsilon^{-8md} \log^{4md}(N\varphi^{-1}) \log^{4md+m}(N\varepsilon^{-1} \log(N\varphi^{-1})))$ time.

Remark 2. If we do not know an upper bound δ on $ERR_{\mathcal{S}_{P_i}}(\mathcal{F}_{\square}^d)$, then by slightly modifying the data structure above we can get the following result. Let $ERR_{\mathcal{S}_{P_i}}(\mathcal{F}_{\square}^d) \leq \delta_i$ for unknown parameters $\delta_i \in [0, 1)$. Having the same complexities as in Theorem C.8, we report a set of indexes J such that, if $a_{\theta(\ell)} + \varepsilon + \delta_i \leq \mathcal{M}_{R(\ell)}(P_i) \leq b_{\theta(\ell)} - \varepsilon - \delta_i$ for every $\ell \in [m]$, then $i \in J$, and if $j \in J$, then $a_{\theta(\ell)} - \varepsilon - \delta_j \leq \mathcal{M}_{R(\ell)}(P_j) \leq b_{\theta(\ell)} + \varepsilon + \delta_j$, for every $\ell \in [m]$.

D Missing details from Section 5

PROOF OF LEMMA 5.3. We have $|C| = O(\varepsilon^{-d+1})$ and each value $\gamma_{\vec{v}}^{(i)}$ is computed in $\Lambda_{\mathcal{S}_{P_i}}$ time. Hence all points $\bigcup_{\vec{v} \in C} \Gamma_{\vec{v}}$ are computed in $O(\varepsilon^{-d+1} \cdot \sum_{i \in [N]} \Lambda_{\mathcal{S}_{P_i}})$ time. Furthermore, each tree $\mathcal{T}_{\vec{v}}$ is computed in $O(N \log N)$ time and it uses $O(N)$ space. Hence, the data structure has $O(\varepsilon^{-d+1} N)$ space and can be constructed in $O(\varepsilon^{-d+1} \cdot \sum_{i \in [N]} \Lambda_{\mathcal{S}_{P_i}} + \varepsilon^{-d+1} N \log N)$ time. $\square$

Remark 1. The data structure in Theorem 5.4 can be made dynamic under insertion or deletion of synopses. If a synopsis $\mathcal{S}_P$ is inserted/deleted we can update our data structure in $O(\Lambda_{\mathcal{S}_P} + \log N)$.

Remark 2. If we do not know an upper bound δ on $ERR_{\mathcal{S}_{P_i}}(\vec{\mathcal{F}}_k^d)$, then by slightly modifying the data structure from Theorem 5.4 we can get the following result. Let $ERR_{\mathcal{S}_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta_i$ for unknown parameters $\delta_i \in [0, 1)$. Having the same complexities as in Theorem 5.4, we report a set of indexes J such that, if $\mathcal{M}_{\vec{u},k}(P_i) \geq a_{\theta} + \varepsilon + \delta_i$, then $i \in J$, and if $j \in J$, then $\mathcal{M}_R(P_j) \geq a_{\theta} - \varepsilon - \delta_j$. In other words, we might miss some datasets that satisfy the predicate $PRED_{\mathcal{M}_{\vec{u},k}, \theta}$ close to the boundary a_{θ} . More specifically, we might miss an index i if $\mathcal{M}_{\vec{u},k}(P_i) \in [a_{\theta}, a_{\theta} + \varepsilon + \delta_i)$ and we always report an index $j \in J$ if $\mathcal{M}_{\vec{u},k}(P_j) \geq a_{\theta} - \varepsilon - \delta_j$.

Remark 3. Our data structure in Theorem 5.4 straightforwardly satisfies $O(\log N)$ delay.

D.1 Approximate data structure for the Pref problem with a logical expression

So far, we assumed that the (query) logical expression Π contains only one predicate, $PRED_{\mathcal{M}_{\vec{u},k}, \theta}$. In this subsection, we extend our data structure to work for any constant number m of predicates. Handling disjunction of predicates is straightforward using the designed data structure, as we had for the Ptile problem. On the other hand, it is not clear how to handle conjunction of predicates. In this section, we describe a data structure that handles m conjunctions of predicates.

Main idea. The main difference from the previous subsection is that we construct a data structure for each subset of m vectors in the ε -net to compute an approximate solution for the Pref problem.

We construct an ε -net and for every vector $\vec{v}$ in the ε -net we use $\mathcal{S}_{P_i}$ to get a value that approximates $\omega_k(P_i, \vec{v})$, for every $i \in [N]$. For every subset of m vectors in C , and every index $i \in [N]$ we define a point in $\mathbb{R}^m$ whose coordinates are the computed values on the subset of vectors. We store

these points to a range tree. When we get query with m query vectors, we find the closest vectors from the ε -net and we use the corresponding range tree to identify the datasets whose scores lie in $\theta^{(1)}, \dots, \theta^{(m)}$.

Data structure. We construct an ε -net C on $\mathbb{R}^d$. For every $\vec{v} \in C$ and every $i \in [N]$, we get a value $\gamma_{\vec{v}}^{(i)}$ from S_{P_i} as an approximation to $\omega_k(P_i, \vec{v})$. Let $\Gamma_{\vec{v}} = \{\gamma_{\vec{v}}^{(i)} \mid i \in [N]\}$. For every subset $V = \{\vec{v}_1, \dots, \vec{v}_m\}$ of m vectors in C , let $B_V = \{(\gamma_{\vec{v}_1}^{(i)}, \dots, \gamma_{\vec{v}_m}^{(i)}) \mid i \in [N]\}$ be a set of N points in m dimensions. For every subset V , we construct the (static) m -dimensional range tree $\mathcal{T}_V$ for reporting queries over B_V .

Query procedure. Let $\vec{u}^{(1)}, \dots, \vec{u}^{(m)}$ be the query vectors and $a_{\theta^{(1)}}, \dots, a_{\theta^{(m)}}$ be the thresholds. For every $\ell \in [m]$, we go through each vector in C and we compute $\vec{v}_\ell = \arg \min_{\vec{h} \in C} \|\vec{u}^{(\ell)} - \vec{h}\|$. Let $V = \{\vec{v}_1, \dots, \vec{v}_m\}$ and let $R = [a_{\theta^{(1)}} - \varepsilon, 1] \times \dots \times [a_{\theta^{(m)}} - \varepsilon, 1]$ be a rectangle in $\mathbb{R}^m$. We run a reporting query on $\mathcal{T}_V$ using the query rectangle R . Let J be the set of all indexes reported.

Correctness. Let $\Pi = \text{PRED}_{\mathcal{M}_{\vec{u}^{(1)}, k}, \theta^{(1)}} \wedge \dots \wedge \text{PRED}_{\mathcal{M}_{\vec{u}^{(m)}, k}, \theta^{(m)}}$ be the logical expression consisting of the conjunction of m range-predicates. Recall that $\mathbf{q}_\Pi(\mathcal{P})$ contains denotes the correct set of indexes we should report.

LEMMA D.1. $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$ and for every $j \in J$, $\omega_k(P_j, \vec{u}^{(\ell)}) \geq a_{\theta^{(\ell)}} - 2\varepsilon - 2\delta$, for every $\ell \in [m]$.

The correctness follows by straightforwardly extending the proof of Lemma 5.2.

Analysis.

LEMMA D.2. The data structure has $O(\varepsilon^{-md+m} N \log^{m-1} N)$ space and can be constructed in $O(\varepsilon^{-d+1} \sum_{i \in [N]} \Lambda_{S_{P_i}} + \varepsilon^{-md+m} N \log^{m-1} N)$ time.

PROOF. We have $|C| = O(\varepsilon^{-d+1})$ and each value $\gamma_{\vec{v}}^{(i)}$ is computed in $\Lambda_{S_{P_i}}$ time. Hence all points $\bigcup_{\vec{v} \in C} \Gamma_{\vec{v}}$ are computed in $O(\varepsilon^{-d+1} \cdot \sum_{i \in [N]} \Lambda_{S_{P_i}})$ time. Furthermore, each tree $\mathcal{T}_V$ is computed in $O(N \log^{m-1} N)$ time and it uses $O(N \log^{m-1} N)$ space. There are $O(\varepsilon^{-md+m})$ subsets of m vectors in C . □

LEMMA D.3. For $m > 1$, the query time is $O(m\varepsilon^{-d+1} + \log^{m-1}(N) + |J|)$.

PROOF. The query procedure spends $O(m \cdot \varepsilon^{-d+1})$ time to compute $V = \{\vec{v}_1, \dots, \vec{v}_m\}$. Then, it runs a reporting query on $\mathcal{T}_V$ that takes $O(\log^{m-1}(N) + |J|)$ time. The result follows. □

We set $\varepsilon \leftarrow \varepsilon/2$ and we get the final result.

THEOREM D.4. Let $\{S_{P_1}, \dots, S_{P_N}\}$ be the input to the Pref problem, such that S_{P_i} is a synopsis of a dataset $P_i \subset \mathbb{R}^d$, where $d \geq 1$ is a constant, with $\text{ERR}_{S_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta$ for $\delta \in [0, 1)$ and $k \geq 1$, for every $i \in [N]$. Let $\varepsilon \in (0, 1)$ and $m > 1$ be two parameters. A data structure of size $O(\varepsilon^{-md+m} N \log^{m-1} N)$ can be constructed in $O(\varepsilon^{-d+1} \sum_{i \in [N]} \Lambda_{S_{P_i}} + \varepsilon^{-md+m} N \log^{m-1} N)$ time, such that, given m query unit vectors $\vec{u}^{(1)}, \dots, \vec{u}^{(m)} \in \mathbb{R}^d$ and m thresholds $a_{\theta^{(1)}}, \dots, a_{\theta^{(m)}} \geq 0$, it returns a set of indexes J such that $\mathbf{q}_\Pi(\mathcal{P}) \subseteq J$, where $\Pi = \text{PRED}_{\mathcal{M}_{\vec{u}^{(1)}, k}, \theta^{(1)}} \wedge \dots \wedge \text{PRED}_{\mathcal{M}_{\vec{u}^{(m)}, k}, \theta^{(m)}}$, and for every $j \in J$, $\mathcal{M}_{\vec{u}^{(\ell)}, k}(P_j) \geq a_{\theta^{(\ell)}} - \varepsilon - \delta$ for every $\ell \in [m]$. The query time is $O(m\varepsilon^{-d+1} + \log^{m-1}(N) + |J|)$.

While in Theorem D.4 we only mention conjunction of predicates, the same result holds if we have any disjunction/conjunction of m predicates. If we assume that ε is an arbitrarily small constant and m is a constant then the data structure has size $O(N \log^{m-1} N) = \tilde{O}(N)$, can be constructed in

$O(\sum_{i \in [N]} \Lambda_{S_{P_i}} + N \log^{m-1} N) = \tilde{O}(\sum_{i \in [N]} \Lambda_{S_{P_i}} + N)$ time, and it has $O(\log^{m-1}(N) + |J|) = \tilde{O}(1 + |J|)$ query time.

Remark 1. The data structure can be made dynamic under insertion or deletion of synopses. If a synopsis S_P is inserted/deleted we can update our data structure in $O(\varepsilon^{-d+1} \Lambda_{S_P} + \varepsilon^{-md+m} \log^m N)$ time.

Remark 2. If we do not know an upper bound δ on $\text{ERR}_{S_{P_i}}(\vec{\mathcal{F}}_k^d)$, then by slightly modifying the data structure above we can get the following result. Let $\text{ERR}_{S_{P_i}}(\vec{\mathcal{F}}_k^d) \leq \delta_i$ for unknown parameters $\delta_i \in [0, 1)$. Having the same complexities as in Theorem C.8, we report a set of indexes J such that, if $\mathcal{M}_{\vec{u}^{(\ell)}, k}(P_i) \geq a_{\theta^{(\ell)}} + \varepsilon + \delta_i$ for every $\ell \in [m]$, then $i \in J$, and if $j \in J$, then $\mathcal{M}_{\vec{u}^{(\ell)}, k}(P_j) \geq a_{\theta^{(\ell)}} - \varepsilon - \delta_j$, for every $\ell \in [m]$.